音廻 ／ コード解説（行番号つき）

攻撃・盾・カメラ・ノーツの向きのしくみ

叩いた時の破片とレーザー、着弾、敵のダメージと撃破、ロングの盾、必殺技に加えて、カメラの動きと、ノーツが流れてくる向き（上から・横から）を変える仕組みを、ファイルと行番号つきでまとめた資料。企業の方に「これはどこで、どうやっているの？」と聞かれた時に、すぐ該当のコードを開いて答えられるようにしている。

2026年11月10日時点の move_lane ブランチのコードを読んで作成。行番号はこの時点のもの（コードを書き足すとずれる）。

---

1 全体の地図

2 カメラ：2層の描画・譜面でカメラを動かす・揺れと見上げ

3 ノーツが流れてくる向き：上から・横から・判定円の移動

4 敵：HP・ダメージ・撃破・光らせる

5 判定からイベントへ：OnJudge と GameEvent

6 叩いた時の攻撃：破片とレーザー（HitBurstFx）

7 着弾（ImpactFx）と、今は止めている攻撃の光（AttackOrbs）

8 ロングの盾：張る・ヒビ・割れる（LongShield）

9 必殺技：ゲージ・弱点・ビーム

10 全部を支える道具：光の帯・形・頂点の送り方

11 読んでいて気になった所

コードの見方：灰色の枠の上に「ファイル名　開始〜終了行」を書いている。長い関数は一部を「(〇〜〇行は省略)」にしている。色付きの行は、説明の中で特に大事な行。

1. 全体の地図

処理は、いくつものクラスに分かれている。分け方の決まりは一つで、**「見た目だけのクラス」と「ダメージや音を決める ChartObject」を分ける**こと。見た目のクラスは「何かが起きた」をコールバックかイベント（GameEvent）で知らせるだけで、ダメージは入れない。

ファイル	担当	主な関数（行）
`Game.cpp`	オブジェクトを2層（背景・譜面）に分けて描く	`Game.cpp : 125〜159行`
`Camera.cpp/.h`	カメラ。背景の層だけ見上げ・振り向きを足す	`Camera.cpp : 116〜186行`
`CameraTrack.h`	カメラの名前（プリセット）と位置の表	`CameraTrack.h : 25〜51行`
`Stage1Scene.cpp`	毎フレーム、譜面のカメラをカメラに渡す	`Stage1Scene.cpp : 112〜185行`
`LaneTrack.h`	レーンの向き（奥・上・右・左）と区切り	`LaneTrack.h : 19〜35行`
`ChartLoader.cpp`	json の tracks・camera を読む	`ChartLoader.cpp : 73行`
`ChartObject.cpp`	司令塔。判定・ダメージ・音・向きの計算・カメラの区切り	`ChartObject.cpp : 1232〜1260行`
`Enemy.cpp/.h`	敵のHP、瀕死・撃破・爆発、揺れと光	`Enemy.h : 25〜36行`
`HitBurstFx.cpp/.h`	叩いたノーツの破片と、拍で飛ぶレーザー	`HitBurstFx.cpp : 13〜33行`
`ImpactFx.cpp/.h`	着弾の輪・六角形・火花	`ImpactFx.cpp : 144〜186行`
`AttackOrbs.cpp/.h`	判定円から敵へ飛ぶ光（今は止めている）	`AttackOrbs.cpp : 33〜53行`
`LongShield.cpp/.h`	ロングの盾。張る・ヒビ・割れる・波紋・破片	`LongShield.cpp : 345〜473行`
`SpecialMove / WeakPoint / SpecialBeam`	必殺技の状態・弱点・ビーム	`SpecialMove.cpp : 22〜47行`
`GameEvent.h`	「叩いた」「拍の頭」「当たった」などの放送局	全体（84行）
`LongBeamRibbon / RibbonShapes`	点の並びを光の帯にして描く道具	`LongBeamRibbon.cpp : 29〜86行`

叩いてから敵が光るまで

JudgeSystem::RegisterHit
判定してコンボを数える

→

ChartObject::OnJudge
ダメージを入れる／GameEvent::OnJudge を放送

→

HitBurstFx
（聞いている）その場で破片がはじける

↓ 拍の頭（ChartObject::Update が GameEvent::OnBeat を放送）

HitBurstFx
破片が発射位置へ飛び、敵へまっすぐレーザー

→

届いた
GameEvent::OnAttackHit を放送

→

ChartObject
（聞いている）盾があれば波紋、なければ ImpactFx ＋ 敵を光らせる

ロングを押してから盾が割れるまで

先端が判定ラインに届く
UpdateLongShield → BeginLong

→

LongShield
盾が張られる（張られている間、叩いたダメージは止まる）

↓ 曲がり角と終点ごとに Checkpoint

成功
ヒビが1本入る

→

失敗
盾が灰色になり、もう割れない

→

ヒビが必要な数に届く
Break → 破片 → OnBreak → ダメージと大きな演出

カメラと向きが決まるまで

w001_lane.json
"camera" と "tracks"

→

ChartLoader::Load
CameraKey と LaneTrack に読む

→

ChartObject::LoadChart
判定円の移動・ノーツの出現位置を、読み込みの時に1回だけ計算

↓ 毎フレーム

Stage1Scene::Update
GetCameraPose → SetFixedPose

→

Game::Draw
背景の層 → 奥行きを消す → 譜面の層

→

ChartObject::Draw
向きに合わせてノーツを回す・判定ラインをつなぐ・キーの画像

2. カメラ

カメラは1つだけ。それを「背景の層」と「譜面の層」で2回使い分けて描いている。カメラの位置は、譜面（json）の小節の区切りで決まり、毎フレーム計算される。

2-1. 2層で描く（Game::Draw）

Game.cpp　125〜144行　—　描く順番

```
125 void Game::Draw()
126 {
127     // 描画前処理
128     //Renderer::DrawStart();
129     m_Instance->m_PostFlash.BeginScene();
130 
131     Camera* cam = &m_Instance->m_Camera;
132 
133     //1回目に背景描画　SetWorldOffsetの影響を受ける
134     cam->SetLayer(Camera::Layer::World);
135     for (auto& o : m_Instance->m_Objects)
136         if (o->GetLayer() == Camera::Layer::World)o->Draw(cam);
137 
138     //背景の奥行を消す　譜面は背景より手前
139     Renderer::ClearCurrentDepth();
140 
141     //2回目　譜面描画
142     cam->SetLayer(Camera::Layer::Playfield);
143     for (auto& o : m_Instance->m_Objects)
144         if (o->GetLayer() == Camera::Layer::Playfield)o->Draw(cam);
```

- オブジェクトは `GetLayer()` で、自分が背景（World）か譜面（Playfield）かを答える。何も書かなければ背景。ChartObject・HitBurstFx・PauseOverlay などは Playfield を返す。
- 背景を全部描いた後に `ClearCurrentDepth` で奥行きの記録を消す。これで、譜面は背景のどんな物より必ず手前に描かれる。

> **なぜこうしているか：**背景を動かす演出（見上げる・振り向く）を入れても、ノーツと判定円の位置は1ドットもずれない。企業の意見「ノーツの周りは見やすく」を守りながら、背景だけ派手に動かせる。

2-2. 背景の層だけ見上げる（Camera::SetCamera）

Camera.cpp　116〜150行　—　3Dの時

```
116 void Camera::SetCamera(int mode)
117 {
118     //3D
119     if (mode == 0)
120     {
121         Vector3 shakenPos = m_Position + m_ShakeOffset;
122         Vector3 shakenTarget = m_Target + m_ShakeOffset;
123         // ビュー変換行列作成
124         Vector3 eye = m_Position + m_ShakeOffset;
125         Vector3 target = m_Target + m_ShakeOffset;
126 
```

```
        if (m_Layer == Layer::World)
        {
            Vector3 forward = target - eye;
            float dist = forward.Length();
            if (dist > 0.0001f)
            {
                forward /= dist;
                Vector3 right = Vector3::Up.Cross(forward);    //左手系
                right.Normalize();
                //右軸回りにマイナス回すと上を向く。その後に左右に振る
                Matrix rot = Matrix::CreateFromAxisAngle(right, -m_WorldPitch)
                    * Matrix::CreateFromAxisAngle(Vector3::Up, m_WorldYaw);
                forward = Vector3::TransformNormal(forward, rot);
                eye += forward * m_WorldDolly;    //前に寄る
                target = eye + forward * dist;
            }
        }
        Vector3 up = Vector3(0.0f, 1.0f, 0.0f);
        m_ViewMatrix = DirectX::XMMatrixLookAtLH(eye, target, up);    //左手系
        Renderer::SetViewMatrix(&m_ViewMatrix);
        //プロジェクション行列の生成
        constexpr float fieldOfView = DirectX::XMConvertToRadians(45.0f); // 視野角

        float aspectRatio = static_cast<float>(Application::GetWidth()) / static_cast<float>(Application::GetHeight());
// アスペクト比
```

- 揺れ（`m_ShakeOffset`）は、目の位置と見る先の両方に同じだけ足す。両方を同じだけずらすので、向きは変わらず、画面全体が小刻みにずれる。
- `m_Layer == World` の時だけ、見る向き（`forward`）を「画面の右の軸」まわりに `m_WorldPitch`（上を向く）、真上の軸まわりに `m_WorldYaw`（左右）回し、`m_WorldDolly` だけ前に寄る。
- 画面の右の軸は `Vector3::Up.Cross(forward)`（真上と前の外積）で求める。左手系なので、この順番で右になる。

2-3. ロングを押し切った時・盾が割れた時に見上げる

ChartObject.cpp　1207〜1222行　—　ChartObject::Update の最後

```
    //Camera::Layer::Worldのほうの目線をロングを押し切った瞬間に上げて一拍後戻す
    {
        float beat = (m_Bpm > 0.0f) ? 60.0f / m_Bpm : 0.5f;
        float a = (m_MusicTime - m_GazeKickTime) / beat;    //経過を拍で数える
        float k = 0.0f;
        if (a >= 0.0f && a <= 1.0f)
        {
            float up = std::min(a / 0.15f, 1.0f);    //0.15拍で上がり切る
            float down = 1.0f - std::max((a - 0.4f) / 0.6f, 0.0f);    //0.4拍から戻る
            k = up * down;
            k = k * k * (3.0f - 2.0f * k);    //smoothstep 動き始めと終わりを滑らかに
        }
        Game::GetInstance()->GetCamera()->SetWorldOffset(
            0.0f, DirectX::XMConvertToRadians(3.0f) * k, 0.0f
        );
    }
```

盾が割れた時（`ChartObject.cpp : 280行`）などで `m_GazeKickTime` に今の時刻を入れる。Update は「そこから何拍たったか」で、0.15拍で3度上を向き、0.4拍から戻り始め、1拍で元に戻る量 `k` を計算して `SetWorldOffset` に渡す。smoothstep（`k×k×(3−2k)`）で、動き始めと止まる所をなめらかにしている。

> **なぜこうしているか：**時間（秒）ではなく拍で数えるので、BPM が違う曲でも「1拍で見上げて戻る」という、曲に合った動きになる。

2-4. 画面の揺れ（ScreenEffects）

ScreenEffects.cpp　9〜18行　—　揺れを起こす

```
 9 void ScreenEffects::TriggerScreenShake(float duration, float strength)
10 {
11     //既に揺れている場合長い方を優先する
12     if (duration > m_ShakeDuration - m_ShakeTimer)
13     {
14         m_ShakeDuration = duration;
15         m_ShakeTimer = duration;
16     }
17     m_ShakeStrength = std::max(m_ShakeStrength, strength);
18 }
```

ScreenEffects.cpp　26〜51行　—　揺れの計算

```
26 void ScreenEffects::Update(float deltaTime)
27 {
28     //画面シェイク
29     if (m_ShakeTimer > 0.0f)
30     {
31         m_ShakeTimer -= deltaTime;
32         if (m_ShakeTimer < 0.0f) m_ShakeTimer = 0.0f;
33 
34         //残り時間の割合で強度を減衰させる(終盤ほど揺れが小さくなる)
35         float t = (m_ShakeDuration > 0.0001f) ? (m_ShakeTimer / m_ShakeDuration) : 0.0f;
36         float currentStrength = m_ShakeStrength * t;
37 
38         float ox = ((float)rand() / RAND_MAX * 2.0f - 1.0f) * currentStrength;
39         float oy = ((float)rand() / RAND_MAX * 2.0f - 1.0f) * currentStrength;
40         m_ShakeOffset = Vector3(ox, oy, 0.0f);
41 
42         if (m_ShakeTimer <= 0.0f)
43         {
44             m_ShakeStrength = 0.0f;
45             m_ShakeOffset = Vector3::Zero;
46         }
47     }
48     else
49     {
50         m_ShakeOffset = Vector3::Zero;
51     }
```

残り時間の割合で強さを下げながら、毎フレームランダムなずれを作る。ChartObject::Draw の最初で `ChartObject.cpp : 1755行` でカメラに渡している。すでに揺れている時は「長い方・強い方」を残すので、小さな揺れで大きな揺れが消えない（Enemy の Flash と同じ考え方）。

2-5. 譜面のカメラの区切り：json

charts/w001_lane.json　3〜13行　—　w001_lane.json の "camera"

```
3 "camera":[
4 {"fromMeasure":1,"preset":"far","blendBeats":4},
5 {"fromMeasure":10,"preset":"low","blendBeats":4},
```

```
{"fromMeasure":21,"preset":"default","blendBeats":2},
{"fromMeasure":23,"preset":"rightup","blendBeats":8},
{"fromMeasure":26,"preset":"leftup","blendBeats":8},
{"fromMeasure":29,"preset":"far","blendBeats":4},
{"fromMeasure":37,"preset":"low","blendBeats":4},
{"fromMeasure":42,"preset":"high","blendBeats":8},
{"fromMeasure":48,"preset":"far","blendBeats":4}
],
```

項目	意味
`fromMeasure`	何小節目の頭で、そのカメラに着くか
`preset`	カメラの名前。CameraTrack.h の表から探す。表にない名前（"default" など）は、いつものカメラ
`blendBeats`	何拍かけて動くか。小節の頭で動き終わるように、その拍数だけ前から動き始める

2-6. カメラの名前と位置（CameraTrack.h）

CameraTrack.h　25〜51行　—　CameraPresets::Find

```
    inline bool Find(const std::string& name, CameraPose& out)
    {
        using DirectX::SimpleMath::Vector3;
        struct Preset { const char* name; Vector3 eye; Vector3 target; };
        //判定円の真ん中が (0, -480, -210) の時に合わせた値
        static const Preset kPresets[] =
        {
            { "front",   Vector3(   0.0f,  150.0f, -2000.0f), Vector3(  0.0f,  150.0f,   0.0f) },    //真正面（横・上から来るレーン用）
            { "low",     Vector3(   0.0f,  250.0f, -1250.0f), Vector3(  0.0f, -430.0f, 450.0f) },    //低い所から　奥が遠くまで見える
            { "far",     Vector3(   0.0f,  850.0f, -1350.0f), Vector3(  0.0f, -480.0f, 350.0f) },    //少し引いた
            { "high",    Vector3(   0.0f, 1300.0f,  -800.0f), Vector3(  0.0f, -480.0f, 450.0f) },    //高い所から見下ろす
            { "right",   Vector3( 520.0f,  560.0f,  -900.0f), Vector3( 60.0f, -480.0f, 350.0f) },    //右から
            { "left",    Vector3(-520.0f,  560.0f,  -900.0f), Vector3(-60.0f, -480.0f, 350.0f) },    //左から
            { "rightup", Vector3( 420.0f, 1300.0f,  -900.0f), Vector3( 40.0f, -480.0f, 400.0f) },    //右上から（rightとhighを混ぜた）
            { "leftup",  Vector3(-420.0f, 1300.0f,  -900.0f), Vector3(-40.0f, -480.0f, 400.0f) },    //左上から
        };
        for (const auto& p : kPresets)
        {
            if (name == p.name)
            {
                out.eye = p.eye;
                out.target = p.target;
                return true;
            }
        }
        return false;    //知らない名前は"default"
    }
```

- 名前と「目の位置」「見る先」の組を、表（配列）にして並べている。新しいカメラを足したい時は、この表に1行足すだけで json から使える。
- 全部、判定円の真ん中が (0, −480, −210) の時に合わせた値。判定ラインと、ノーツが出てくる奥の端の両方が画面に入るように計算してある。
- "default" は表にない。いつものカメラは Stage1Scene が決める（次の項）。

2-7. 読み込み（ChartLoader.cpp）

ChartLoader.cpp　107〜120行　—　"camera" を読む

```
107         //カメラの区切り　なければ今のカメラを使用
108         if (root.Has("camera") && root["camera"].IsArray())
109         {
110             for (const auto& c : root["camera"].arrayValue)
111             {
112                 CameraKey key;
113                 key.fromMeasure = c["fromMeasure"].AsInt(1);
114                 key.preset = c["preset"].AsString("default");
115                 key.blendBeats = c["blendBeats"].AsFloat(2.0f);
116                 result.cameraKeys.push_back(key);
117             }
118             std::sort(result.cameraKeys.begin(), result.cameraKeys.end(),
119                 [](const CameraKey& a, const CameraKey& b) { return a.fromMeasure < b.fromMeasure; });
120         }
```

項目がなければ既定値（1小節目、"default"、2拍）。最後に小節の順に並べ替えておくので、後の計算は前から順に見るだけで済む。

2-8. 今の時刻のカメラを出す（ChartObject::GetCameraPose）

ChartObject.cpp　608〜640行

```
608 CameraPose ChartObject::GetCameraPose(const CameraPose& defaultPose) const
609 {
610     if (m_CameraKeys.empty()) return defaultPose;
611 
612     auto poseOf = [&](const std::string& name)
613         {
614             CameraPose p;
615             return CameraPresets::Find(name, p) ? p : defaultPose;
616         };
617 
618     float beat = (m_Bpm > 0.0f) ? 60.0f / m_Bpm : 0.5f;    //1拍の秒数
619     float bar = beat * (float)m_BeatsPerBar;    //1小節の秒数
620 
621     CameraPose cur = defaultPose;    //最初の区切りより前は今のカメラ
622     for (const auto& k : m_CameraKeys)
623     {
624         float endT = m_Offset + (k.fromMeasure - 1) * bar;    //この小節の頭で動き終わる
625         float startT = endT - k.blendBeats * beat;    //ここから動き始める
626         CameraPose next = poseOf(k.preset);
627 
628         if (m_MusicTime >= endT) { cur = next; continue; }    //動き終わったなら次に
629         if (m_MusicTime > startT && endT - startT > 0.00001f)
630         {
631             float t = (m_MusicTime - startT) / (endT - startT);    //経過時間を01で
632             t = t * t * (3.0f - 2.0f * t);    //スムーズ
633             cur.eye = Vector3::Lerp(cur.eye, next.eye, t);
634             cur.target = Vector3::Lerp(cur.target, next.target, t);
635         }
636         break;    //まだ来ていない区切りから先は見ない
637     }
638 
639     return cur;
640 }
```

1. 区切りを前から順に見る。「その区切りの小節の頭の時刻」が `endT`、そこから `blendBeats` 拍前が `startT`。

2. 今が `endT` を過ぎていれば、カメラはもうその区切りの位置。次の区切りへ進む。
3. 今が `startT` と `endT` の間なら、前のカメラと次のカメラを、経過の割合 t（smoothstep でなめらかに）で混ぜる（`Lerp`）。
4. まだ来ていない区切りが出てきたら、そこで止める（`break`）。

小節 → 秒は、`offset + (小節 − 1) × 1小節の秒数`。1小節の秒数は `60 ÷ BPM × 1小節の拍数`（w001 なら 0.375 × 4 = 1.5秒）。

2-9. 毎フレーム、カメラに渡す（Stage1Scene）

Stage1Scene.cpp 24〜31行 — いつものカメラ

```
24 CameraPose Stage1Scene::DefaultCameraPose()
25 {
26     Vector3 c = m_Chart->GetJudgeLineCenter();
27     CameraPose p;
28     p.eye = kUseFixedEye ? m_FixedEye : c + Vector3(0.0f, m_CamHeight, -m_CamBack);
29     p.target = c + Vector3(0.0f, 0.0f, m_CamLookAhead);
30     return p;
31 }
```

Stage1Scene.cpp 147〜151行 — Stage1Scene::Update の中

```
147     //譜面のカメラの区切りに合わせてカメラを動かす
148     {
149         CameraPose p = m_Chart->GetCameraPose(DefaultCameraPose());
150         Game::GetInstance()->GetCamera()->SetFixedPose(p.eye, p.target);
151     }
```

Camera.cpp 197〜204行 — 位置と見る先をそのまま決める

```
197 void Camera::SetFixedPose(DirectX::SimpleMath::Vector3 pos, DirectX::SimpleMath::Vector3 target)
198 {
199     m_Position = pos;
200     m_Target = target;
201     m_Fixed = true;
202     // derive spherical params from provided pose so toggle/controls are smooth
203     DirectX::SimpleMath::Vector3 d = m_Position - m_Target;
204     m_distance = std::sqrt(d.x * d.x + d.y * d.y + d.z * d.z);
```

> **なぜこうしているか：**カメラの位置を「今の曲の時刻から毎回計算する」形にしている。前のフレームの位置を少しずつ動かす形だと、ポーズや処理落ちでずれが積み重なる。時刻から毎回計算すれば、いつ見ても譜面どおりの位置になる。Camera::Update は固定モード（`m_Fixed`）なら何もしないので、デバッグ用の操作とぶつからない。

3. ノーツが流れてくる向き

「奥から」だけだったノーツを、小節ごと・レーンごとに「上から」「右から」「左から」に変えられる仕組み。判定円の位置も区間ごとに動かせる。今の w001_lane では、10〜20小節目と37〜38小節目が「上から」。横（縦の列）は試した結果、キーとの対応が取りにくいので外した。

3-1. json の書き方

charts/w001_lane.json　14〜21行　—　w001_lane.json の "tracks"

```
14 "tracks":[
15   {"key":0,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]},
16   {"key":1,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]},
17   {"key":2,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]},
18   {"key":3,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]},
19   {"key":4,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]},
20   {"key":5,"sections":[{"fromMeasure":10,"dir":"top"},{"fromMeasure":21,"dir":"back"},{"fromMeasure":37,"dir":"top"},
{"fromMeasure":39,"dir":"back"}]}
21 ],
```

項目	意味
`key`	どのレーンか（0〜5 ＝ ＳＤＦＪＫＬ）
`sections`	区切りの並び。次の区切りまで同じ設定が続く。最初の区切りより前は「奥から」
`dir`	`back`（奥）、`top`（上）、`right`（右）、`left`（左）
`judge`	（あれば）その区間の判定円の位置。ゲームの中の座標で書く

3-2. 向きと区切りの形（LaneTrack.h）

LaneTrack.h　1〜57行　—　LaneDir・TrackSection・LaneTrack・FromString・SpawnOffset

```
1  #pragma once
2  #include <vector>
3  #include <string>
4  #include <SimpleMath.h>
5
6  //ノーツが流れてくる方向
7  enum class LaneDir { Back, Top, Right, Left};
8
9  //レーンの区切り：「この小節からこの方向に」
10 struct TrackSection
11 {
12     int fromMeasure = 1;    //この小節から
13     LaneDir dir = LaneDir::Back;    //この方向に
14     bool hasJudge = false;    //判定円の位置が書いてあるか
15     DirectX::SimpleMath::Vector3 judge;    //この区間の判定円の位置
16 };
```

```

//レーン一本。今はキーとレーンが同じ番号に
struct LaneTrack
{
    int key = 0;    //このレーンのキー番号
    std::vector<TrackSection> sections;    //fromMeasureの小さい順

    //measure小節目の設定（区切りがなければ奥）
    TrackSection SectionAt(int measure) const
    {
        TrackSection s;
        for (const auto& sec : sections)
        {
            if (sec.fromMeasure <= measure) s = sec;
            else break;
        }
        return s;
    }
};

namespace LaneDirUtil
{
    inline LaneDir FromString(const std::string& s)
    {
        if (s == "top") return LaneDir::Top;
        if (s == "right") return LaneDir::Right;
        if (s == "left") return LaneDir::Left;
        return LaneDir::Back;
    }

    //判定円から見て出現位置をどれだけずらすか（奥からは今まで通りなので使わない）
    inline DirectX::SimpleMath::Vector3 SpawnOffset(LaneDir d)
    {
        switch (d)
        {
        case LaneDir::Top: return DirectX::SimpleMath::Vector3(0.0f, 535.0f, 150.0f);
        case LaneDir::Right: return DirectX::SimpleMath::Vector3(900.0f, 200.0f, 0.0f);
        case LaneDir::Left: return DirectX::SimpleMath::Vector3(-900.0f, 0.0f, 0.0f);
        case LaneDir::Back: return DirectX::SimpleMath::Vector3(0.0f, 0.0f, 0.0f);
        }
    }
```

- `SectionAt(小節)`：区切りを前から見て、その小節以前の最後の区切りを返す。区切りが小節の順に並んでいる前提（読み込みで並べ替えている）。
- `SpawnOffset`：判定円から見て、出現位置をどれだけずらすか。上なら上と少し奥（y 535、z 150）、右なら +x、左なら −x。

LaneTrack.h　59〜92行　—　向きごとの見た目の決まり

```

    //向きごとのノーツの回し方
    inline DirectX::SimpleMath::Matrix NoteRotation(LaneDir d)
    {
        using DirectX::SimpleMath::Matrix;
        switch (d)
        {
        case LaneDir::Top:    //起こして面をカメラへ向ける
            return Matrix::CreateRotationX(-DirectX::XM_PIDIV2);
        case LaneDir::Right:
        case LaneDir::Left:    //起こしてさらに90度回転
            return Matrix::CreateRotationX(-DirectX::XM_PIDIV2) * Matrix::CreateRotationZ(DirectX::XM_PIDIV2);
        default:
            return Matrix::Identity;
        }
```

```
    }

    //判定円が並ぶ向き（横からくるレーンは縦に）
    inline DirectX::SimpleMath::Vector3 Lateral(LaneDir d)
    {
        if (d == LaneDir::Right || d == LaneDir::Left) return DirectX::SimpleMath::Vector3(0.0f, 1.0f, 0.0f);
        return DirectX::SimpleMath::Vector3(1.0f, 0.0f, 0.0f);
    }

    //キーの画像を判定円からどれだけずらすか
    inline DirectX::SimpleMath::Vector2 IconOffset(LaneDir d)
    {
        switch (d)
        {
        case LaneDir::Right: return DirectX::SimpleMath::Vector2(-90.0f, 0.0f);    //右から来る->画像は左
        case LaneDir::Left:  return DirectX::SimpleMath::Vector2(90.0f, 0.0f);    //左から来る->画像は右
        case LaneDir::Top:   return DirectX::SimpleMath::Vector2(0.0f, -90.0f);    //上から来る->画像は下
        default:             return DirectX::SimpleMath::Vector2(0.0f, 0.0f);
        }
    }
```

関数	使う所	すること
`NoteRotation`	ノーツを描く時	上から：x 軸まわりに −90度起こして、面をカメラへ。横から：さらに z 軸まわりに 90度（縦長）
`Lateral`	判定ラインを描く時	判定円が並ぶ向き。横から来るレーンは縦（0,1,0）
`IconOffset`	キーの画像	ノーツが来ない側へ 90ピクセルずらして出す

3-3. 読み込み（ChartLoader.cpp）

ChartLoader.cpp　73〜105行　—　"tracks" を読む

```
        //レーンの区切り（なければ6本ともずっと奥から）
        result.tracks.resize(6);
        for (int i = 0; i < 6; i++) result.tracks[i].key = i;
        if (root.Has("tracks") && root["tracks"].IsArray())
        {
            for (const auto& t : root["tracks"].arrayValue)
            {
                int key = t["key"].AsInt(-1);
                if (key < 0 || key >= 6) continue;
                auto& track = result.tracks[key];
                track.sections.clear();
                if (t.Has("sections") && t["sections"].IsArray())
                {
                    for (const auto& s : t["sections"].arrayValue)
                    {
                        TrackSection sec;
                        sec.fromMeasure = s["fromMeasure"].AsInt(1);
                        sec.dir = LaneDirUtil::FromString(s["dir"].AsString("back"));
                        //判定円の位置
                        if (s.Has("judge"))
                        {
                            sec.hasJudge = true;
                            sec.judge = Vector3(s["judge"]["x"].AsFloat(0.0f),
                                s["judge"]["y"].AsFloat(0.0f),
                                s["judge"]["z"].AsFloat(0.0f));
                        }
```

```
                        track.sections.push_back(sec);
                    }
                    std::sort(track.sections.begin(), track.sections.end(),
                        [](const TrackSection& a, const TrackSection& b) { return a.fromMeasure < b.fromMeasure; });
                }
            }
        }
```

レーンは必ず6本作り（`resize(6)`）、json に書いてあるレーンだけ区切りを上書きする。書いていないレーンは、区切りなし＝ずっと奥から。`"judge"` があった区切りだけ `hasJudge = true`。

3-4. 読み込みの時に1回だけ計算する（ChartObject::LoadChart）

ChartObject.cpp　457〜520行　—　順番が大事

```
    m_JudgeCircles = result.judgeCircles;

    //レーンの区切りを先に受け取る（判定円を動かすのに使用）
    m_Tracks = result.tracks;
    m_CameraKeys = result.cameraKeys;
    m_Offset = result.offset;
    //区切りに判定円の位置があればその区間だけ判定円をそこへ動かす
    //奥からくるノーツも動いた判定円から決めたいからkHighwayModeより前で実行
    ApplyTrackJudgePositions(result.bpm, result.beatsPerBar);

    if (kHighwayMode)
    {
        for (auto& note : m_Notes)
        {
            if (note.IsFlick()) continue;    //フリックは敵側へ飛ぶので今まで通り
            if (note.lane < 0 || note.lane >= (int)m_JudgeCircles.size()) continue;

            float arrive = note.IsLong() ? note.arrivalTime : note.time;    //判定んに届く時間
            Vector3 judge = m_JudgeCircles[note.lane].GetPosition(arrive);
            note.spawnPos = judge + Vector3(0.0f, 0.0f, m_HighwayDepth);    //同じ高さで奥から
            note.spawnTime = arrive - m_HighwayApproach;    //全ノーツ同じ速さで
            note.controlPoints.clear();    //曲線じゃなくて直線で
            note.spiralRadius = 0.0f;    //螺旋もやめる
        }
    }

…  （483〜516行は省略）

    //レーンの方向（上から横から）に合わせてノーツの出現位置を決め直す
    //判定ガイドのトラックの向きは今まで通りにしたいのでその計算の後でやる
    ApplyLaneDirections(result.bpm, result.beatsPerBar);
```

1. **判定円を動かす**（ApplyTrackJudgePositions）。
2. **奥からの出現位置**を、動いた後の判定円から決める（kHighwayMode のブロック）。
3. 判定ガイドの道の向きを、奥からの位置で平均しておく（省略した所）。
4. **上・横の出現位置**に書き換える（ApplyLaneDirections）。

> **なぜこうしているか：**どれも「判定円の位置」から計算するので、判定円を先に動かしておく必要がある。上・横の書き換えを最後にしているのは、判定ガイドの道（奥へ伸びる線）の向きを、今まで通り奥向きのままにしたいから。

3-5. 出現位置を書き換える（ApplyLaneDirections）

ChartObject.cpp 1406〜1425行

```
1406 void ChartObject::ApplyLaneDirections(float bpm, int beatsPerBar)
1407 {
1408     float secPerBar = (bpm > 0.0f ? 60.0f / bpm : 0.5f) * (float)beatsPerBar;    //1小節の時間
1409
1410     for (auto& n : m_Notes)
1411     {
1412         if (n.IsFlick() || n.IsLong()) continue;    //現在対応ノーツ以外はスルー
1413         if (n.lane < 0 || n.lane >= (int)m_Tracks.size()) continue;
1414
1415         //判定の時刻が何小節目か（offsetの拍が1小節目の頭）
1416         int measure = (int)std::floor((n.time - m_Offset) / secPerBar + 1);
1417         LaneDir dir = m_Tracks[n.lane].SectionAt(measure).dir;
1418         n.flowDir = dir;    //描くときに回す
1419         if (dir == LaneDir::Back) continue;    //奥からは今の出現位置をキープ
1420
1421         Vector3 judge = m_JudgeCircles[n.lane].GetPosition(n.time);    //届く瞬間の判定円
1422         n.spawnPos = judge + LaneDirUtil::SpawnOffset(dir);
1423         n.controlPoints.clear();    //まっすぐ降ってくる
1424     }
1425 }
```

ノーツの判定の時刻が何小節目かを計算し、その小節のレーンの向きを調べる。奥以外なら、出現位置（`spawnPos`）を「判定の瞬間の判定円 ＋ SpawnOffset」に書き換え、曲線の制御点を消して直線にする。向きは `flowDir` に覚えておき、描く時にノーツを回すのに使う。

なぜ出現位置を変えるだけで、上から降ってくるのか（NotePath.cpp）

NotePath.cpp 18〜52行 — タップの位置の計算

```
18     Vector3 EvaluateNotePosition(const ChartNote& note, float now)
19     {
20         float startTime = note.GetSpawnTime();
21         float endTime = note.time;
22         float duration = endTime - startTime;
23         if (duration < 0.0001f) duration = 0.0001f; // ゼロ除算防止
24
25         float t = (now - startTime) / duration;
26         if (t < 0.0f) t = 0.0f;
27         if (t > 1.0f) t = 1.0f;
28
29         float highSpeed = Game::GetInstance()->GetHighSpeed();
30         //const auto& p0 = note.spawnPos;
31         const auto& p3 = note.judgePos;
32         const auto& p0 = p3 + (note.spawnPos - p3) * highSpeed;    //出現距離を変更
33
34         // 制御点なし → 直線補間
35         if (note.controlPoints.empty())
36         {
37             return Vector3::Lerp(p0, p3, t);
38         }
39         // 制御点1個 → 2次ベジェ
40         if (note.controlPoints.size() == 1)
41         {
42             const auto& p1 = note.controlPoints[0];
43             return Vector3::Lerp(Vector3::Lerp(p0, p1, t), Vector3::Lerp(p1, p3, t), t);
44         }
45         // 制御点2個 → 3次ベジェ（De Casteljauアルゴリズム）
46         const auto& p1 = note.controlPoints[0];
```

```
        const auto& p2 = note.controlPoints[1];
        auto a = Vector3::Lerp(p0, p1, t);
        auto b = Vector3::Lerp(p1, p2, t);
        auto c = Vector3::Lerp(p2, p3, t);
        return Vector3::Lerp(Vector3::Lerp(a, b, t), Vector3::Lerp(b, c, t), t);
    }
```

タップは「出現時刻から判定時刻まで、出現位置 p0 から判定位置 p3 へまっすぐ動く」作り。だから出現位置を真上にすれば、上から降ってくる。ハイスピードは「判定位置からの距離」を伸ばしているだけなので、上からのノーツにもそのまま効く。

> **今できていないこと**
> ロングとフリックは `continue` で飛ばしているので、いつも奥から来る。ロングの帯は `ChartObject.cpp : 643〜662行` で「判定円の位置 ＋ 奥行き（z）」として描いているため、向きを変えるには、ここを「向きのベクトル」で伸ばす形に直す必要がある（レーン拡張の設計案の、ファーガソン曲線の話）。

3-6. 判定円を区間ごとに動かす（ApplyTrackJudgePositions）

ChartObject.cpp 1428〜1479行

```
1428 void ChartObject::ApplyTrackJudgePositions(float bpm, int beatsPerBar)
1429 {
1430     float beat = (bpm > 0.0f) ? 60.0f / bpm : 0.5f;    //一拍の秒数
1431     float bar = beat * (float)beatsPerBar;    //一小節の秒数
1432     constexpr float kMoveBeats = 4.0f;        //何拍かけて動くか
1433     float moveSec = beat * kMoveBeats;
1434 
1435     for (int lane = 0; lane < (int)m_Tracks.size() && lane < (int)m_JudgeCircles.size(); lane++)
1436     {
1437         const LaneTrack& track = m_Tracks[lane];
1438 
1439         bool any = false;    //このレーンに位置の指定があるか
1440         for (const auto& s : track.sections) if (s.hasJudge) any = true;
1441         if (!any) continue;
1442 
1443         const ChartJudgeCircle original = m_JudgeCircles[lane];    //コピー
1444         auto measureOf = [&](float t) { return (int)std::floor((t - m_Offset) / bar) + 1; };
1445         //t秒の位置：その小節の区切りに位置があればそこに、なければ元の動きに
1446         auto posAt = [&](float t)
1447             {
1448                 TrackSection s = track.SectionAt(measureOf(t));
1449                 return s.hasJudge ? s.judge : original.GetPosition(t);
1450             };
1451 
1452         std::vector<ChartCircleKeyframe> keys;
1453         auto push = [&](float t, const Vector3& p)
1454             {
1455                 ChartCircleKeyframe k;
1456                 k.time = t;
1457                 k.x = p.x;
1458                 k.y = p.y;
1459                 k.z = p.z;
1460                 keys.push_back(k);
1461             };
1462 
1463         //元のキーフレームは位置の指定がない区間の物だけ残す
1464         for (const auto& k : original.keyframe)
1465             if (!track.SectionAt(measureOf(k.time)).hasJudge) keys.push_back(k);
```

```
1466
1467        //区切りのところ：一拍前->小節の頭
1468        for (const auto& s : track.sections)
1469        {
1470            float T = m_Offset + (s.fromMeasure - 1) * bar;
1471            push(T - moveSec, posAt(T - moveSec));
1472            push(T, posAt(T + 0.001f));    //ちょうど境目だと前の小節と判断
1473        }
1474
1475        std::sort(keys.begin(), keys.end(),
1476            [](const ChartCircleKeyframe& a, const ChartCircleKeyframe& b) { return a.time < b.time; });
1477        m_JudgeCircles[lane].keyframe = keys;
1478    }
1479 }
```

判定円は「何秒にどこにいるか」の予定表（キーフレーム）で動いている。予定表の読み方は次の関数。

ChartObject.cpp　86〜122行　—　予定表の前後を線でつなぐ

```
86  Vector3 ChartJudgeCircle::GetPosition(float now) const
87  {
88      if (keyframe.empty()) return Vector3::Zero;
89      if (keyframe.size() == 1)
90          return Vector3(keyframe[0].x, keyframe[0].y, keyframe[0].z);
91
92      // nowが最初のキーフレームより前 → 先頭の座標に固定
93      if (now <= keyframe.front().time)
94      {
95          const auto& k = keyframe.front();
96          return Vector3(k.x, k.y, k.z);
97      }
98      // nowが最後のキーフレームより後 → 末尾の座標に固定
99      if (now >= keyframe.back().time)
100     {
101         const auto& k = keyframe.back();
102         return Vector3(k.x, k.y, k.z);
103     }
104
105     // nowを挟む2つのキーフレームを探して線形補間
106     for (size_t i = 0; i + 1 < keyframe.size(); i++)
107     {
108         const auto& a = keyframe[i];
109         const auto& b = keyframe[i + 1];
110         if (now >= a.time && now <= b.time)
111         {
112             float span = b.time - a.time;
113             float t = (span > 0.00001f) ? (now - a.time) / span : 0.0f;
114             Vector3 pa(a.x, a.y, a.z);
115             Vector3 pb(b.x, b.y, b.z);
116             return LerpVector3(pa, pb, t);
117         }
118     }
119     // ここに来ることは理論上ないが、保険として末尾を返す
120     const auto& k = keyframe.back();
121     return Vector3(k.x, k.y, k.z);
122 }
```

ApplyTrackJudgePositions は、この予定表に「区切りの1拍前（前の区間の場所）」と「区切りの小節の頭（新しい区間の場所）」の2つを足す。GetPosition がその間を線でつなぐので、1拍かけて判定円がすっと動く。区切りの境目ちょうどは小数の誤差で前の小節と判断されることがあるので、0.001秒だけ後ろを見ている。

> **なぜこうしているか：**予定表を書き換えるだけなので、判定・ノーツの行き先・判定ライン・攻撃の光の出どころなど、判定円の位置を使っている全部の所が、何も変えずに一緒に動く。

3-7. 描く時（ChartObject::Draw）

ChartObject.cpp 1481〜1487行 — その時刻のレーンの向き

```
1481 LaneDir ChartObject::LaneDirAt(int lane, float time) const
1482 {
1483     if (lane < 0 || lane >= (int)m_Tracks.size()) return LaneDir::Back;
1484     float bar = ((m_Bpm > 0.0f) ? 60.0f / m_Bpm : 0.5f) * (float)m_BeatsPerBar;
1485     int measure = (int)std::floor((time - m_Offset) / bar) + 1;
1486     return m_Tracks[lane].SectionAt(measure).dir;
1487 }
```

ChartObject.cpp 1869〜1883行 — ノーツ1つの位置・色・大きさ・回し方

```
1869     auto noteVisual = [&](const ChartNote& note, const NoteLook& look, NoteDrawContext& out) -> bool
1870         {
1871             if (note.judged) return false;
1872             float st = note.GetSpawnTime();
1873             float et = note.time;
1874             if (m_MusicTime < st || m_MusicTime > et) return false;    // 表示範囲外
1875 
1876             Vector3 c = note.chargeShot ? Vector3(1.0f, 0.8f, 0.25f) : GetLaneColor(note.lane);
1877             out.pos = NotePath::EvaluateNotePosition(note, m_MusicTime);    // Long ではない方
1878             out.color = c + (Vector3(1.0f, 1.0f, 1.0f) - c) * look.whiteMix;    // 白を混ぜる
1879             out.scale = m_NoteScale * look.scale;
1880             out.musicTime = m_MusicTime;    // 足す
1881             out.rot = LaneDirUtil::NoteRotation(note.flowDir);    //向きに合わせて回転
1882             return true;
1883         };
```

ノーツを描く時のワールド行列は「大きさ × 回し方 × 位置」（`ChartObject.cpp : 1911行`、トレースは TraceNote.cpp の Draw）。奥からは回さない（単位行列）。

ChartObject.cpp 1772〜1792行 — 判定ラインに向きを渡す

```
1772     //判定ライン・判定バッド・レーンのトラック・拍の線
1773     //ノーツとかより先に描く
1774     if ((int)m_JudgeCircles.size() >= JudgeLineGuide::kLaneCount)
1775     {
1776         JudgeLineGuide::Frame gf = {};
1777         float highSpeed = Game::GetInstance()->GetHighSpeed();    //ノーツと同じくハイスピで奥行を伸ばす
1778         for (int i = 0; i < JudgeLineGuide::kLaneCount; i++)
1779         {
1780             LaneDir dir = LaneDirAt(i, m_MusicTime);
1781             gf.judgePos[i] = m_JudgeCircles[i].GetPosition(m_MusicTime);
1782             //奥からは今まで通り。横・上は、ノーツが来る方へ線を流す
1783             gf.trackOffset[i] = (dir == LaneDir::Back ? m_LaneTrackOffset[i] : LaneDirUtil::SpawnOffset(dir)) *
highSpeed;
1784             gf.lateral[i] = LaneDirUtil::Lateral(dir);
1785             gf.group[i] = (int)dir;
1786             gf.laneColor[i] = GetLaneColor(i);
1787             gf.laneGlow[i] = m_LaneGlow[i];
1788         }
1789         gf.musicTime = m_MusicTime;
1790         gf.bpm = m_Bpm;
```

```
1791        gf.beatsPerBar = m_BeatsPerBar;
1792        gf.approachSeconds = m_TrackApproachSeconds;
```

JudgeLineGuide.cpp　117〜131行　—　同じ向きで隣り合うレーンだけ線でつなぐ

```
117     auto drawRuns = [&](const Vector3* p, float width, const GuideParams& gp)
118         {
119             int a = 0;
120             for (int i = 1; i <= n; i++)
121             {
122                 if (i < n && f.group[i] == f.group[a]) continue;
123                 int b = i - 1;    //aからbまでが1つのまとまり
124                 Vector3 head = (b > a) ? (p[a + 1] - p[a]) * 0.5f : latelar[a] * 45.0f;
125                 Vector3 tail = (b > a) ? (p[b] - p[b - 1]) * 0.5f : latelar[b] * 45.0f;
126                 DrawRibbon(p[a] - head, p[a], width, camPos, gp);
127                 for (int k = a; k < b; k++) DrawRibbon(p[k], p[k + 1], width, camPos, gp);
128                 DrawRibbon(p[b], p[b] + tail, width, camPos, gp);
129                 a = i;
130             }
131         };
```

判定ライン（判定円をつなぐ光の線）と拍の線は、`group`（向きの番号）が同じで隣り合うレーンだけをつなぐ。向きが違うレーンの間には線を引かない。横から来るレーンは `lateral` が縦なので、判定円の板も縦に並ぶ。

ChartObject.cpp　2056〜2072行　—　キーの画像（どこを叩くか）

```
2056    //奥から以外のレーンは、判定円の横にキーの画像を出す（どこを叩くかわかるように）
2057    {
2058        constexpr float kIconSize = 70.0f;
2059        float beat = (m_Bpm > 0.0f) ? 60.0f / m_Bpm : 0.5f;
2060        Renderer::SetBlendState(BS_ALPHABLEND);
2061        for (int lane = 0; lane < 6; lane++)
2062        {
2063            LaneDir dir = LaneDirAt(lane, m_MusicTime + beat);    //判定円が動き始めた時から出す
2064            if (dir == LaneDir::Back) continue;
2065            Vector2 off = LaneDirUtil::IconOffset(dir);
2066            m_LaneIcons[lane].SetPosition(judgeCircleScreenPos[lane] + Vector3(off.x, off.y, 0.0f));
2067            m_LaneIcons[lane].SetScale(kIconSize, kIconSize, 1.0f);
2068            m_LaneIcons[lane].SetAlpha(1.0f);
2069            m_LaneIcons[lane].Draw(cam);
2070        }
2071        Renderer::SetBlendState(BS_NONE);
2072    }
```

奥以外のレーンには、判定円の横にキーの画像（S〜L）を出す。1拍先の向きで判断しているので、判定円が動き始めた時から出る。

横（縦の列）をやめた理由

キーボードは横一列なので、判定円も横一列なら「見た位置 ＝ 押す指」がそのまま対応する。縦の列にすると、毎回「上から何番目 → どのキー」と頭で置き換える必要があり、0.1秒単位の反応が追いつかない。上からは判定円が横一列のままなので使える。横から来るなら、端のレーン（S は左の外から、L は右の外から）だけにする案が残っている。

4. 敵：HP・ダメージ・撃破・光らせる（Enemy）

4-1. HP と状態

状態	いつ	見た目（Enemy::Update / Draw）
Alive	最初	ゆっくり回る
Dying（瀕死）	HP が `kDyingHp`（1）まで減った	0.6秒ごとに赤く点滅。回転がゆっくりになる
Defeated（とどめ）	最後のノーツの判定が終わった時、瀕死なら `Finish()`	真っ白に光り、だんだん強く揺れる
（爆発）	とどめから `kExplodeTime`（0.9秒）	消えて、`OnExploded` で大きな着弾・揺れ・閃光

Enemy.h　25〜36行　—　ダメージを受ける

```
25      void Damage(float amount)
26      {
27          if (m_State == State::Defeated) return; //倒された後の攻撃は無効
28          m_Hp -= amount;
29          if (m_Hp < kDyingHp) m_Hp = kDyingHp;     //とどめの前は瀕死
30          m_HitTimer = 0.25f;
31          if (m_State == State::Alive && m_Hp <= kDyingHp)
32          {
33              m_State = State::Dying;     //瀕死
34              if (m_OnDying) m_OnDying();
35          }
36      }
```

ChartObject.cpp　1192〜1198行　—　とどめ（ChartObject::Update）

```
1192    //全部のノーツの判定が終わったら最後のノーツをとどめノーツにする
1193    if (!m_FinishChecked && m_AudioStarted && m_TotalNote > 0 && m_Notes.empty())
1194    {
1195        m_FinishChecked = true;
1196        if (m_EnemyRef && m_EnemyRef->IsDying()) m_EnemyRef->Finish();
1197        else;     //瀕死まで行けなかった時の演出追加用
1198    }
```

Enemy.cpp　131〜139行　—　とどめを刺す

```
131 void Enemy::Finish()
132 {
133     if (m_State == State::Defeated) return;     //2回目はなにもしない
134     m_State = State::Defeated;
135     m_Hp = 0.0f;
136     m_DefeatTime = 0.0f;
137     Flash(Vector3(1.0f, 1.0f, 1.0f), 1.0f, 0.15f);     //とどめの瞬間真っ白
138     if (m_OnDefeated) m_OnDefeated();
139 }
```

> **なぜこうしているか：**HP は 1 より下がらないので、どれだけ攻撃しても曲の途中で倒れない。倒れるのは「最後のノーツを叩き終えた時」だけ。ダメージの量をどう調整しても、**撃破は必ず曲の終わり**に起きる。音ゲーとして、曲の途中で戦いが終わるのを防いでいる。

4-2. 状態ごとの動き（Enemy::Update）

Enemy.cpp　37〜71行

```
37 void Enemy::Update()
38 {
39     const float dt = 1.0f / 60.0f;
40 
41     //回転：瀕死・倒れた後はゆっくり
42     m_Rotation.y += (m_State == State::Alive) ? 0.01f : 0.003f;
43 
44     if (m_HitTimer > 0.0f) m_HitTimer -= dt;
45     if (m_FlashTime > 0.0f) m_FlashTime = std::max(0.0f, m_FlashTime - dt);
46 
47     //瀕死0.6sごとに赤く点滅
48     if (m_State == State::Dying)
49     {
50         m_DyingBlink -= dt;
51         if (m_DyingBlink <= 0.0f)
52         {
53             Flash(Vector3(1.0f, 0.2f, 0.15f), 0.5f, 0.25f);
54             m_DyingBlink = 0.6f;
55         }
56     }
57 
58     //とどめのあと白くなって爆発
59     if (m_State == State::Defeated && !m_Exploded)
60     {
61         m_DefeatTime += dt;
62         float k = std::min(m_DefeatTime / kExplodeTime, 1.0f);
63         Flash(Vector3(1.0f, 1.0f, 1.0f), 0.3f + 0.7f * k, 0.05f);    //ちょっとずつ白く
64         if (m_DefeatTime >= kExplodeTime)
65         {
66             m_Exploded = true;
67             if (m_OnExploded) m_OnExploded();
68         }
69     }
70 
71 }
```

瀕死の点滅は 0.6秒ごとに赤い Flash を呼ぶだけ。とどめの後は、経過の割合 k で白く光る強さを上げていき、`kExplodeTime` で爆発を知らせる。

4-3. 入るダメージの一覧（HP は 100）

いつ	どこ	量	条件
叩くたび	`ChartObject.cpp : 1240行`	`60 ÷ m_TotalNote`（全部叩くと合計 60）	盾が張られている間は入らない
ロングを押し切った	`ChartObject.cpp : 185〜192行`	`40 ÷ ロングの本数`	最後まで押していた時
盾が割れた	`ChartObject.cpp : 272〜282行`	`40 ÷ ロングの本数 × その盾のロングの本数`	ヒビが全部入った時
必殺技	`ChartObject.cpp : 286〜311行`	`15 × (0.3 + 0.7 × チャージ量) × 倍率`	弱点なら ×2.0、防がれたら ×0.3

`m_TotalNote` は「タップの数 ＋ ロングのチェックポイントの数」（`ChartObject.cpp : 436〜569行` の最初）。ダメージは叩いた瞬間に入り、レーザーが当たった時ではない（見た目と計算を分けているため）。

4-4. 敵を一瞬光らせる（Flash）

Enemy.cpp　141〜150行

```
141 void Enemy::Flash(const DirectX::SimpleMath::Vector3& color, float strength, float duration)
142 {
143     //今光っている方が強ければ、そちらを残す（弱い光で強い光を上書きしない）
144     if (strength < CurrentFlash()) return;
145 
146     m_FlashColor = color;
147     m_FlashStrength = strength;
148     m_FlashDuration = std::max(0.01f, duration);
149     m_FlashTime = m_FlashDuration;
150 }
```

Enemy.cpp　153〜158行

```
153 float Enemy::CurrentFlash() const
154 {
155     if (m_FlashTime <= 0.0f) return 0.0f;
156     float k = m_FlashTime / m_FlashDuration;    // 1->0
157     return m_FlashStrength * k * k;
158 }
```

shader/enemyFlash.hlsl　1〜28行　—　元の色と光の色を混ぜる

```
1  #include "common.hlsl"
2  
3  Texture2D g_Texture : register(t0);
4  SamplerState g_SamplerState : register(s0);
5  
6  //敵を一瞬光らせる
7  cbuffer EnemyFlash : register(b6)
8  {
9      float4 flashColor;   //rgb:変える色  a:どれだけ変えるか
10 }
11 
12 float main(in PS_IN input) : SV_Target
```

```
{
    float4 color;
    if (Material.TextureEnable)
    {
        color = g_Texture.Sample(g_SamplerState, input.tex);
        color *= input.col;
    }
    else
    {
        color = input.col * Material.Diffuse;
    }

    // 元の色と光らせたい色を、flashColor.a の割合で混ぜる
    color.rgb = lerp(color.rgb, flashColor.rgb, flashColor.a);
    return color;
}
```

なぜこうしているか：攻撃が連続で当たると Flash が何度も呼ばれる。弱い光（普通の着弾）が強い光（弱点・とどめ）を上書きすると、大事な瞬間が見えなくなる。「今より弱い光は無視する」の1行で、強い演出が必ず残る。2乗で減らすのは、「パッ」と光ってすぐ戻る感じにするため。

5. 判定からイベントへ：OnJudge と GameEvent

5-1. 判定の入口（JudgeSystem）

JudgeSystem.cpp　301〜306行

```
301 void JudgeSystem::RegisterHit(int laneIndex, int rank, bool isLong)
302 {
303     m_Combo++;
304     if (m_Combo > m_MaxCombo) m_MaxCombo = m_Combo;
305     if (m_OnJudge) m_OnJudge(laneIndex, rank, isLong);
306 }
```

タップ・フリック・トレース・ロングのチェックポイント、どれで当たっても最後はここを通る。ロングのチェックポイントだけ `isLong = true` で呼ばれる（`JudgeSystem.cpp : 141行`）。ChartObject は Init で `SetOnJudge` に OnJudge を渡している（`ChartObject.cpp : 146行`）。

5-2. 判定がヒットした時（ChartObject::OnJudge）

ChartObject.cpp　1232〜1260行

```
1232 void ChartObject::OnJudge(int laneIndex, int rank, bool isLong)
1233 {
1234     m_Special.AddHit(rank, false);
1235     m_ComboHud.TriggerComboPop();
1236     if (isLong || kTapBarrier)  m_ComboHud.TriggerBarrier(laneIndex);
1237     if (isLong || kTapRankText) m_ComboHud.TriggerRankPop(laneIndex, rank);
1238 
1239     //叩くたびに少しダメージを与える　ロングの盾が張られている間はダメージなし
1240     if (m_EnemyRef && m_TotalNote > 0 && !m_LongShield.IsUp()) m_EnemyRef->Damage(60.0f / m_TotalNote);
1241 
1242     if (laneIndex < 0 || laneIndex >= 6) return;
1243     m_LaneGlow[laneIndex] = 1.0f;    //判定したレーンのバッドを光らせる
1244     Vector3 pos = m_JudgeCircles[laneIndex].GetPosition(m_MusicTime);
1245 
1246     //「叩いた」を知らせる（破片の演出は、これを聞いて HitBurstFx が出す）
1247     GameEvent::OnJudge().Publish({ laneIndex, rank, pos, GetLaneColor(laneIndex), isLong });
1248 
1249     //判定円から伸びる光
1250     if (isLong || kTapKeyLight)
1251     {
1252         ChartKeyLightEffect le;
1253         le.lane = laneIndex;
1254         le.startTime = m_MusicTime;
1255         le.duration = 0.4f;
1256         le.intensity = 2.5f;
1257         le.origin = pos;
1258         m_LightEffects.push_back(le);
1259     }
1260 }
```

処理	タップ	ロングの途中
必殺技のゲージ・コンボの数字	出る	出る
バリアの光（`kTapBarrier`）	出ない（false）	出る
判定の文字（`kTapRankText`）	設定しだい	出る
ダメージ	入る（盾がなければ）	入る（盾がなければ。実際はいつも盾がある）
GameEvent::OnJudge の放送	する	する（`isLong` 付き）
攻撃の光（AttackOrbs）	出ない	出ない（条件が `!isLong && kTapAttackOrbs`）
判定円から伸びる光の柱	出ない	出ない（条件が `!isLong && kTapKeyLight`）

5-3. GameEvent（放送局）

GameEvent.h　1〜85行　—　全体

```
#pragma once
#include <functional>
#include <vector>
#include <utility>
#include <SimpleMath.h>
#include <winnt.h>

//ゲームでのイベントを知らせる仕組み
namespace GameEvent
{
    //イベントの中身
    struct Judge
    {
        int lane;    //0~5
        int rank;    //0=weak 1=hit 2=graze
        DirectX::SimpleMath::Vector3 pos;    //判定円の位置
        DirectX::SimpleMath::Vector3 color;    //レーンの色
        bool isLong;    //ロングの途中の判定？
    };
    struct Beat    //拍の頭
    {
        long long index;    //曲の頭から何拍めか
        bool isBar;            //小節の頭か
    };
    struct LongEnd { int lane; };    //ロングを最後まで押し切った
    struct ShieldBreak { DirectX::SimpleMath::Vector3 center; };    //ロングの盾が割れた

    struct AttackHit    //攻撃が敵にヒット
    {
        DirectX::SimpleMath::Vector3 pos;    //当たった場所
        DirectX::SimpleMath::Vector3 color;
        int rank;
    };

    //知らせる
    template<class T>
    class Channel
    {
    public:
        using Handler = std::function<void(const T&)>;

        //聞く。戻り値の番号は聞くのをやめるときに使用
```

```
        int Subscribe(Handler h)
        {
            int id = ++m_NextId;
            m_Handlers.push_back({ id,std::move(h) });
            return id;
        }
        //聞くの終了
        void Unsubscribe(int id)
        {
            for(auto it = m_Handlers.begin(); it != m_Handlers.end(); ++it)
                if (it->first == id) { m_Handlers.erase(it); return; }
        }
        //知らせる　聞いている全員の関数を呼ぶ
        void Publish(const T& e) const
        {
            auto copy = m_Handlers;    //呼んだ先で聞くものが増えても壊れないように写しを回すための機構
            for (auto& h : copy) h.second(e);
        }
        void Clear() { m_Handlers.clear(); }

    private:
        std::vector<std::pair<int, Handler>>m_Handlers;
        int m_NextId = 0;
    };

    inline Channel<Judge>& OnJudge() { static Channel<Judge> c; return c; }
    inline Channel<Beat>& OnBeat() { static Channel<Beat> c; return c; }
    inline Channel<LongEnd>& OnLongEnd() { static Channel<LongEnd> c; return c; }
    inline Channel<ShieldBreak>& OnShieldBreak() { static Channel<ShieldBreak> c; return c; }
    inline Channel<AttackHit>& OnAttackHit() { static Channel<AttackHit> c; return c; }

    inline void ClearAll()
    {
        OnJudge().Clear();
        OnBeat().Clear();
        OnLongEnd().Clear();
        OnShieldBreak().Clear();
        OnAttackHit().Clear();
    }
}
```

- `OnJudge()` などは、関数の中の `static` 変数を参照（`&`）で返す。誰が呼んでも同じ1つの放送局になる。
- `Subscribe` で「呼んでほしい関数（ラムダ）」を名簿にしまい、`Publish` で名簿の全員を呼ぶ。名簿の写しを回すので、呼んだ先で登録が増えても壊れない。
- ラムダは `[this]` で自分を覚えている。消える前に `Unsubscribe` し、シーンの始めと終わりに `ClearAll`（`Stage1Scene.cpp : 66行`）で名簿を空にする。

ChartObject.cpp　823〜834行　—　拍の頭を放送する（ChartObject::Update）

```
    //拍の頭を知らせる　1フレームで何拍も進んだ時も飛ばさずに全部知らせるためにwhile
    if (m_Bpm > 0.0f && m_MusicTime >= 0.0f)
    {
        float beatSec = 60.0f / m_Bpm;
        long long beatNow = (long long)std::floor(m_MusicTime / beatSec);
        while (m_LastBeatIndex < beatNow)
        {
            m_LastBeatIndex++;
            bool isBar = (m_BeatsPerBar > 0) && (m_LastBeatIndex % m_BeatsPerBar == 0);
            GameEvent::OnBeat().Publish({ m_LastBeatIndex, isBar });
```

```
833         }
834     }
```

1フレームで何拍も進んだ時（処理落ちなど）も、`while` で1拍ずつ全部知らせる。`if` にすると拍を飛ばしてしまう。

イベント	放送する所	聞いている所
`OnJudge`	ChartObject::OnJudge	HitBurstFx（破片を出す）
`OnBeat`	ChartObject::Update	HitBurstFx（撃つ）
`OnAttackHit`	HitBurstFx::Update（レーザーが届いた）	ChartObject（着弾の演出）
`OnLongEnd`	SetOnLongEnd の中	（今は誰も聞いていない）
`OnShieldBreak`	SetOnBreak の中	（今は誰も聞いていない）

> **なぜこうしているか：**演出のクラスは放送を聞くだけなので、斬撃や破片の演出を足した時も、判定の処理（ChartObject）を変えずに済んだ。やめる時も、Stage1Scene の AddObject の行を消すだけでよい。

6. 叩いた時の攻撃：破片とレーザー（HitBurstFx）

叩いたノーツがその場で判定の色の破片になってはじけ、次の拍の頭で破片の一つ一つが敵のまわりへ飛び、そこからまっすぐなレーザーになって敵に刺さる。Stage1Scene で追加し、狙う場所を渡している（`Stage1Scene.cpp : 89行`）。

6-1. 決めごと（HitBurstFx.h）

HitBurstFx.h　21〜30行　—　判定ごとの色

```
21      static DirectX::SimpleMath::Vector3 RankColor(int rank)
22      {
23          switch (rank)
24          {
25          case 0:  return DirectX::SimpleMath::Vector3(1.0f, 0.82f, 0.30f);    //WEAK：金
26          case 1:  return DirectX::SimpleMath::Vector3(0.30f, 0.85f, 1.0f);    //HIT：水色
27          default: return DirectX::SimpleMath::Vector3(0.0f, 1.0f, 0.0f);    //GRAZE：紫
28          }
29      }
30  private:
```

HitBurstFx.h　86〜101行　—　調整用の値

```
86      static constexpr float kLift = 30.0f;    //判定円からどれだけ上で弾けるか
87      static constexpr float kMinHover = 0.1f;    //弾けてからこれだけ待ってから撃つ
88      static constexpr float kMaxWait = 1.0f;        //拍が来なくてもこの時間待ったら発射
89      static constexpr int   kFireEveryBeats = 1;        //何拍ごとに撃つか（2なら2拍に1回）
90      static constexpr float kFireSpread = 0.4f;        //破片ごとに撃つ時間をずらす最大の秒数（一斉射撃に見せる）
91      static constexpr float kHitFlashTime = 0.10f;    //届いた時に光の玉が広がって消えるまでの秒数
92      static constexpr float kDrag = 9.0f;            //破片の減速（大きいほど早く止まる）
93      static constexpr float kDashTime = 0.12f;        //破片が発射位置へ飛ぶ秒数
94      static constexpr float kLaserTime = 0.18f;        //発射位置から敵に届くまでの秒数
95      static constexpr float kLaserTail = 0.6f;        //レーザーの尾の長さ（発射位置から敵までの割合）
96      static constexpr float kLaunchDistMin = 600.0f;    //発射位置の、敵からの距離
97      static constexpr float kLaunchDistMax = 900.0f;
98      static constexpr int   kMaxBursts = 4;
99      static constexpr int   kMaxLasers = 10;
100 };
101
```

6-2. イベントの聞き方（Init）

HitBurstFx.cpp　13〜33行

```
13  void HitBurstFx::Init()
14  {
15      m_Ribbon.Init();
16      m_Pts.reserve(40);
17      m_Bursts.reserve(kMaxBursts);
18      m_Lasers.reserve(kMaxLasers);
19      m_Rng.seed(std::random_device{}());
20
21      //叩いた：その場で破片になってはじける（ロングの途中の判定は今まで通りなので出さない）
22      m_JudgeSub = GameEvent::OnJudge().Subscribe([this](const GameEvent::Judge& e)
23          {
24              if (e.isLong) return;
```

```
            SpawnBurst(e.pos + Vector3(0.0f, kLift, 0.0f), e.rank);
        });

    //拍の頭：待っている破片を、1つ1つレーザーにして撃つ
    m_BeatSub = GameEvent::OnBeat().Subscribe([this](const GameEvent::Beat& e)
        {
            if (e.index % kFireEveryBeats == 0) FireReady();
        });
}
```

6-3. はじける（SpawnBurst）

HitBurstFx.cpp　41〜63行

```
void HitBurstFx::SpawnBurst(const Vector3& pos, int rank)
{
    if ((int)m_Bursts.size() >= kMaxBursts) m_Bursts.erase(m_Bursts.begin());    //多すぎたら古いものから消す

    Burst b;
    b.rank = rank;
    b.color = RankColor(rank);
    int count = (rank == 0) ? 14 : (rank == 1) ? 10 : 6;    //WEAKほど多い
    float speed = (rank == 0) ? 520.0f : (rank == 1) ? 420.0f : 300.0f;
    for (int i = 0; i < count; i++)
    {
        //上の方へ、ばらばらに飛び散る（レーンの上に落ちてこないように）
        Vector3 dir(Rand(-1.0f, 1.0f), Rand(0.2f, 1.2f), Rand(-0.4f, 0.6f));
        dir.Normalize();
        Piece p;
        p.pos = pos;
        p.vel = dir * speed * Rand(0.6f, 1.2f);
        p.width = Rand(6.0f, 11.0f) * (rank == 0 ? 1.3f : 1.0f);
        p.spin = Rand(0.0f, 6.28f);
        b.piece.push_back(p);
    }
    m_Bursts.push_back(b);
}
```

判定ごとに数と勢いを変える（WEAK 14個・520、HIT 10個・420、GRAZE 6個・300）。向きは、上下の成分を0.2〜1.2 にして、必ず上の方へ飛ばす（レーンの上に落ちてこないように）。

6-4. 漂って拍を待つ → 撃つ（Update・FireReady・FireBurst）

HitBurstFx.cpp　136〜43行　—　Update の前半（破片）

HitBurstFx.cpp　65〜69行

```
void HitBurstFx::FireReady()
{
    for (auto& b : m_Bursts)
        if (b.age >= kMinHover) FireBurst(b);    //はじけた直後のものは次の拍に回す
}
```

HitBurstFx.cpp　71〜80行

```
void HitBurstFx::FireBurst(Burst& b)
{
    for (size_t i = 0; i < b.piece.size(); i++)
```

```
    {
        bool notify = (i == 0);    //当たったを知らせるのは、かたまりの最初の1本だけ
        float delay = notify ? 0.0f : Rand(0.0f, kFireSpread);
        LaunchLaser(b.piece[i].pos, b.color, b.rank, notify, delay);
    }
    b.piece.clear();    //破片はレーザーになったので消す（Updateでかたまりごと片付く）
}
```

- 漂う：速さを毎フレーム減らし（`kDrag`）、少し浮かせ、sin と cos で小さく揺らす。
- 撃つ：拍の頭で、はじけてから `kMinHover` 秒以上たったかたまりだけ。はじけた直後のものは次の拍に回す。拍が来なくても `kMaxWait` 秒で撃つ（保険）。
- 一斉射撃に見せる：破片ごとに 0〜`kFireSpread` 秒の待ちを付ける（`l.age = -delay`。マイナスの間は描かない）。
- 「当たった」はかたまりの最初の1本（`notify`）だけが知らせる。全部で知らせると火花が多すぎて重く、画面もうるさくなる。

6-5. 発射位置の決め方：レーンをまたがない理由（LaunchLaser）

HitBurstFx.cpp　82〜106行

```
void HitBurstFx::LaunchLaser(const Vector3& from, const Vector3& color, int rank, bool notify, float delay)
{
    if ((int)m_Lasers.size() >= kMaxLasers) m_Lasers.erase(m_Lasers.begin());

    Laser l;
    l.from = from;
    l.to = m_Target + Vector3(Rand(-90.0f, 90.0f), Rand(-60.0f, 90.0f), Rand(-40.0f, 40.0f));    //敵の体のあちこち

    //どっちから撃つか：右のレーンの破片は敵の右側から、左は左側から
    //角度は真横の少し下(-15度)〜ほぼ真上(85度)。真下（レーンのある方）からは撃たない
    float side = (from.x >= m_Target.x) ? 1.0f : -1.0f;
    float rad = DirectX::XMConvertToRadians(Rand(-15.0f, 85.0f));
    Vector3 dir(side * std::cos(rad), std::sin(rad), Rand(-0.3f, 0.2f));
    dir.Normalize();
    l.launch = l.to + dir * Rand(kLaunchDistMin, kLaunchDistMax);    //敵のまわりの発射位置

    l.color = color;
    l.rank = rank;
    l.width = (rank == 0) ? 8.0f : (rank == 1) ? 6.0f : 4.5f;
    l.flareSize = (rank == 0) ? 46.0f : (rank == 1) ? 34.0f : 24.0f;    //WEAKほど大きく光る
    l.flareRot = Rand(0.0f, 1.57f);
    l.age = -delay;
    l.notify = notify;
    m_Lasers.push_back(l);
}
```

> **なぜこうしているか：**破片からそのまま敵へ撃つと、線がレーンの上を通ってしまう。そこで一度、敵のまわりの発射位置へ飛ばしてから撃つ。発射位置は「破片と同じ側」で、「真横の少し下（−15度）〜ほぼ真上（85度）」にしか置かない。真下（レーンのある方）は使わないので、レーザーは敵のまわりのいろんな方向から刺さるが、レーンの上は通らない。曲線（ベジェ）を使わず、2点の間の割合（`Lerp`）だけで動かしている。

6-6. 届いたら知らせる（Update の後半）

HitBurstFx.cpp　160〜179行　—　Update の後半（レーザー）

```
    //レーザー：届いたら「当たった」を知らせる
    std::vector<GameEvent::AttackHit> arrived;    //ループの外でまとめて知らせる
    for (auto& l : m_Lasers)
    {
        l.age += dt;
        if (!l.arrived && l.age >= kDashTime + kLaserTime)
        {
            l.arrived = true;
            if (l.notify) arrived.push_back({ l.to, l.color, l.rank });
        }
    }
    m_Lasers.erase(std::remove_if(m_Lasers.begin(), m_Lasers.end(),
        [](const Laser& l)
        {
            float end = kDashTime + std::max(kLaserTime * (1.0f + kLaserTail), kLaserTime + kHitFlashTime);
            return l.age >= end;    //尾まで届いて、光の玉も消えたら消す
        }), m_Lasers.end());

    for (const auto& h : arrived) GameEvent::OnAttackHit().Publish(h);
}
```

届いたものを `arrived` に集めてから、ループが終わった後で放送する。放送を聞いた先で何かが足されても、回している最中のリストが壊れない（AttackOrbs でつまずいた所と同じ理由）。

6-7. 描く（Draw・DrawFlare）

HitBurstFx.cpp　109〜134行　—　Draw

```
void HitBurstFx::DrawFlare(Camera* cam, const Vector3& pos, const Vector3& color, float size, float rot, float alpha)
{
    if (alpha <= 0.0f) return;
    Vector3 right, up;
    RibbonShapes::CameraBasis(pos, cam->GetPosition(), right, up);    //カメラの方を向いた面の右と上
    const Vector3 white(1.0f, 1.0f, 1.0f);
    const Vector3 light = color + (white - color) * 0.5f;    //色に白を混ぜた明るい色

    //真ん中が一番濃く、両端に向かって消える帯を、angleの向きに1本描く
    auto streak = [&](float angle, float len, float width, float intensity, const Vector3& c)
        {
            Vector3 d = right * std::cos(angle) + up * std::sin(angle);
            m_Pts.clear();
            LongBeamRibbon::Point p;
            p.pos = pos - d * len; p.color = RibbonShapes::MakeColor(c, 0.0f);  m_Pts.push_back(p);
            p.pos = pos;           p.color = RibbonShapes::MakeColor(c, alpha); m_Pts.push_back(p);
            p.pos = pos + d * len; p.color = RibbonShapes::MakeColor(c, 0.0f);  m_Pts.push_back(p);
            m_Ribbon.Draw(cam, m_Pts, width, intensity, 0.0f, c);
        };

    streak(rot, size * 0.9f, size * 1.8f, 0.35f, color);               //ぼんやりした光
    streak(rot + 1.5708f, size * 0.9f, size * 1.8f, 0.35f, color);
    streak(rot, size * 2.8f, size * 0.12f, 0.9f, color);               //長い光の筋（横）
    streak(rot + 1.5708f, size * 2.0f, size * 0.12f, 0.9f, color);    //長い光の筋（縦）
    streak(rot, size * 0.2f, size * 0.4f, 1.0f, white);                //小さな白い芯
}
```

HitBurstFx.cpp　109〜134行　—　光の玉（レンズフレア風）

```
void HitBurstFx::DrawFlare(Camera* cam, const Vector3& pos, const Vector3& color, float size, float rot, float alpha)
{
    if (alpha <= 0.0f) return;
    Vector3 right, up;
    RibbonShapes::CameraBasis(pos, cam->GetPosition(), right, up);    //カメラの方を向いた面の右と上
    const Vector3 white(1.0f, 1.0f, 1.0f);
    const Vector3 light = color + (white - color) * 0.5f;    //色に白を混ぜた明るい色

    //真ん中が一番濃く、両端に向かって消える帯を、angleの向きに1本描く
    auto streak = [&](float angle, float len, float width, float intensity, const Vector3& c)
        {
            Vector3 d = right * std::cos(angle) + up * std::sin(angle);
            m_Pts.clear();
            LongBeamRibbon::Point p;
            p.pos = pos - d * len; p.color = RibbonShapes::MakeColor(c, 0.0f);  m_Pts.push_back(p);
            p.pos = pos;           p.color = RibbonShapes::MakeColor(c, alpha); m_Pts.push_back(p);
            p.pos = pos + d * len; p.color = RibbonShapes::MakeColor(c, 0.0f);  m_Pts.push_back(p);
            m_Ribbon.Draw(cam, m_Pts, width, intensity, 0.0f, c);
        };

    streak(rot, size * 0.9f, size * 1.8f, 0.35f, color);                //ぼんやりした光
    streak(rot + 1.5708f, size * 0.9f, size * 1.8f, 0.35f, color);
    streak(rot, size * 2.8f, size * 0.12f, 0.9f, color);                //長い光の筋（横）
    streak(rot + 1.5708f, size * 2.0f, size * 0.12f, 0.9f, color);    //長い光の筋（縦）
    streak(rot, size * 0.2f, size * 0.4f, 1.0f, white);                //小さな白い芯
}
```

帯	長さ	太さ	役目
2本（横・縦）	size × 0.9	size × 1.8	太くて短い → ぼんやりした丸い光
2本（横・縦）	size × 2.8 ／ 2.0	size × 0.12	細くて長い → 十字の光の筋
1本	size × 0.2	size × 0.4	小さな白い芯

向きは `RibbonShapes::CameraBasis` で求めた「画面の右と上」を使うので、どこから見ても十字に見える。

7. 着弾（ImpactFx）と、今は止めている攻撃の光（AttackOrbs）

7-1.「当たった」を聞く所（ChartObject::Init）

ChartObject.cpp 338〜353行

```
338     m_AttackHitSub = GameEvent::OnAttackHit().Subscribe([this](const GameEvent::AttackHit& e)
339         {
340             if (m_LongShield.IsUp())
341             {
342                 m_LongShield.Ripple(e.pos);    //盾にはじかれた
343                 return;
344             }
345             bool critical = (e.rank == 0);
346             if (critical) m_ImpactFx.SpawnWeakHit(e.pos, e.color);
347             else m_ImpactFx.SpawnSmallHit(e.pos, e.color);
348             if (m_EnemyRef)
349             {
350                 if (critical) m_EnemyRef->Flash(Vector3(1.0f, 0.85f, 0.4f), 0.45f, 0.12f);
351                 else          m_EnemyRef->Flash(Vector3(1.0f, 1.0f, 1.0f), 0.2f, 0.08f);
352             }
353         });
```

なぜこうしているか：着弾の演出（ImpactFx）、盾（LongShield）、敵（Enemy）を持っているのが ChartObject だから。HitBurstFx はそれらを知らないまま「当たった」を放送し、知っている ChartObject が受け取って演出を出す。放送する側と聞く側が逆なだけで、同じ仕組み。

7-2. ImpactFx：輪と火花

ImpactFx.cpp 34〜39行

```
34 void ImpactFx::SpawnSmallHit(const Vector3& pos, const Vector3& color)
35 {
36     Vector3 c = color * kSmallHitBright;
37     AddRing(pos, c, 0, 8.0f, 55.0f, 0.22f, 7.0f);                              // 小さい輪
38     AddSparks(pos, c, 5, 200.0f, 420.0f, 0.25f, 5.0f, Vector3::Zero, 0.0f);    // 少しの火花
39 }
```

ImpactFx.cpp 41〜47行

```
41 void ImpactFx::SpawnWeakHit(const Vector3& pos, const Vector3& color)
42 {
43     const Vector3 gold(1.0f, 0.85f, 0.4f);
44     AddRing(pos, gold, 0, 10.0f, 110.0f, 0.30f, 10.0f);                        // 金の輪
45     AddRing(pos, color, 6, 20.0f, 75.0f, 0.28f, 7.0f, Rand(0.0f, 1.0f));       // 六角形
46     AddSparks(pos, gold, 12, 300.0f, 700.0f, 0.35f, 6.0f, Vector3::Up, 0.2f);  // 上向きに多めの火花
47 }
```

ImpactFx.cpp 49〜60行

```
49 void ImpactFx::SpawnCritical(const Vector3& pos, const Vector3& color, float power)
50 {
51     power = std::clamp(power, 0.0f, 1.0f);
```

```
    const Vector3 gold(1.0f, 0.85f, 0.4f);
    const Vector3 white(1.0f, 1.0f, 1.0f);

    AddRing(pos, white, 0, 10.0f, 80.0f + 60.0f * power, 0.16f, 30.0f);                    // 白い閃光（すぐ消える）
    AddRing(pos, gold, 0, 20.0f, 200.0f + 220.0f * power, 0.45f, 14.0f);                    // 衝撃の輪
    AddRing(pos, color, 6, 30.0f, 130.0f + 120.0f * power, 0.35f, 9.0f, Rand(0.0f, 1.0f));    // 六角形
    AddSparks(pos, gold, 18 + (int)(30.0f * power), 450.0f, 1100.0f, 0.5f, 8.0f, Vector3::Up, 0.25f);
    AddSparks(pos, color, 10 + (int)(10.0f * power), 300.0f, 750.0f, 0.4f, 6.0f, Vector3::Zero, 0.0f);
}
```

ImpactFx.cpp　62〜68行

```
void ImpactFx::SpawnBlocked(const Vector3& pos, const Vector3& fromDir)
{
    const Vector3 shield(0.55f, 0.75f, 1.0f);    // 青白い盾
    AddRing(pos, shield, 6, 70.0f, 95.0f, 0.45f, 12.0f);            // 盾（六角形が出て、少し広がって消える）
    AddRing(pos, shield, 6, 40.0f, 60.0f, 0.30f, 6.0f, 0.5236f);    // 内側の六角形（30度回す）
    AddSparks(pos, shield, 16, 300.0f, 700.0f, 0.35f, 5.0f, -fromDir, 0.75f);    // 来た方へはね返る
}
```

ImpactFx.cpp　124〜139行　—　動かす

```
void ImpactFx::Update(float deltaTime)
{
    for (auto& r : m_Rings) r.age += deltaTime;
    m_Rings.erase(std::remove_if(m_Rings.begin(), m_Rings.end(),
        [](const Ring& r) { return r.age >= r.life; }), m_Rings.end());

    for (auto& s : m_Sparks)
    {
        s.age += deltaTime;
        s.vel.y -= kGravity * deltaTime;                        // 落ちる
        s.vel *= std::max(0.0f, 1.0f - kDrag * deltaTime);    // だんだん遅くなる
        s.pos += s.vel * deltaTime;
    }
    m_Sparks.erase(std::remove_if(m_Sparks.begin(), m_Sparks.end(),
        [](const Spark& s) { return s.age >= s.life; }), m_Sparks.end());
}
```

ImpactFx.cpp　144〜186行　—　描く

```
void ImpactFx::Draw(Camera* cam)
{
    if (m_Rings.empty() && m_Sparks.empty()) return;

    cam->SetCamera(0);
    Renderer::SetBlendState(BS_ADDITIVE);
    const Vector3 camPos = cam->GetPosition();

    // ―― 輪・六角形：最初に速く広がって、だんだんゆっくり。広がりながら薄くなる ――
    for (const auto& r : m_Rings)
    {
        float k = std::clamp(r.age / r.life, 0.0f, 1.0f);
        float e = 1.0f - (1.0f - k) * (1.0f - k) * (1.0f - k);
        float radius = r.r0 + (r.r1 - r.r0) * e;
        float fade = 1.0f - k;

        Vector3 right, up;
        RibbonShapes::CameraBasis(r.pos, camPos, right, up);
        m_Pts.clear();
        Color c = RibbonShapes::MakeColor(r.color, fade);
        if (r.sides >= 3) RibbonShapes::Polygon(m_Pts, r.pos, right, up, radius, r.sides, r.rot, c);
        else              RibbonShapes::Arc(m_Pts, r.pos, right, up, radius, 0.0f, 2.0f * RibbonShapes::kPi, 40, c);
```

```
166
167         m_Ribbon.Draw(cam, m_Pts, r.width * (1.0f - 0.6f * k), 1.3f, r.age, r.color * 0.8f);
168     }
169
170     // ―― 火花：飛んでいる向きに尾を引く短い線 ――
171     for (const auto& s : m_Sparks)
172     {
173         float fade = 1.0f - std::clamp(s.age / s.life, 0.0f, 1.0f);
174         Vector3 tail = s.pos - s.vel * kStreak;
175
176         m_Pts.clear();
177         LongBeamRibbon::Point p;
178         p.pos = tail;                   p.color = RibbonShapes::MakeColor(s.color, 0.0f);         m_Pts.push_back(p);
179         p.pos = (tail + s.pos) * 0.5f;  p.color = RibbonShapes::MakeColor(s.color, 0.6f * fade);  m_Pts.push_back(p);
180         p.pos = s.pos;                  p.color = RibbonShapes::MakeColor(s.color, fade);         m_Pts.push_back(p);
181
182         m_Ribbon.Draw(cam, m_Pts, s.width, 1.5f, s.age, s.color);
183     }
184
185     Renderer::SetBlendState(BS_NONE);
186 }
```

- 輪：最初に速く広がって、だんだんゆっくり（`1−(1−k)³`）。広がりながら薄く、細くなる。
- 火花：重力で落ち、空気の抵抗で遅くなる。尾は「今の位置 − 速さ × 0.035秒」なので、速い火花ほど長い線になる。
- 上限（`kMaxRings`=32、`kMaxSparks`=240）を超えたら古いものから消す（`ImpactFx.cpp : 73〜89行`）。

7-3. AttackOrbs：判定円から敵へ飛ぶ光（今は止めている）

以前の攻撃の光。OnJudge の条件が `(!isLong && kTapAttackOrbs)` で、`kTapAttackOrbs = false` なので、今は一度も飛ばない。仕組みだけ残しておく。

AttackOrbs.cpp　33〜53行　―　3次ベジェの4点を決める

```
33 void AttackOrbs::Launch(int lane, const Vector3& from, const Vector3& to, const Vector3& color, bool critical)
34 {
35     if ((int)m_Orbs.size() >= kMaxOrbs) m_Orbs.erase(m_Orbs.begin());    // 一番古いものを消す
36
37     // 左手のレーンは左へ、右手のレーンは右へ。外側のレーンほど大きくふくらむ
38     float side = (lane < 3) ? -1.0f : 1.0f;
39     float outer = 0.6f + 0.4f * std::fabs(lane - 2.5f) / 2.5f;
40     float j = Rand(0.85f, 1.15f);    // 毎回少しだけ形を変える
41
42     Orb o;
43     o.lane = lane;
44     o.p0 = from;
45     o.p1 = from + Vector3(side * kSideOut * outer * j, kArcHeight * j, 0.0f);           // 判定円の真上・外側へ
46     o.p2 = to + Vector3(side * kSideOut * 0.35f * outer, kArcHeight * 0.45f * j, 0.0f);    // 敵の少し上・外側から入る
47     o.p3 = to;
48     o.color = color;
49     o.t = 0.0f;
50     o.dur = kFlightTime * Rand(0.9f, 1.1f);
51     o.crit = critical;
52     m_Orbs.push_back(o);
53 }
```

AttackOrbs.cpp　55〜71行　―　届いたらまとめて知らせる

```
55 void AttackOrbs::Update(float deltaTime)
56 {
```

```
    m_Arrived.clear();
    for (auto& o : m_Orbs)
    {
        o.t += deltaTime;
        if (o.t >= o.dur) m_Arrived.push_back(o);
    }
    m_Orbs.erase(std::remove_if(m_Orbs.begin(), m_Orbs.end(),
        [](const Orb& o) { return o.t >= o.dur; }), m_Orbs.end());

    // 着いたものを知らせる（ループの外で呼ぶので、呼んだ先で Launch しても大丈夫）
    if (m_OnArrive)
    {
        for (const auto& o : m_Arrived) m_OnArrive(o.lane, o.p3, o.color, o.crit);
    }
}
```

左手のレーン（ＳＤＦ）は左へ、右手（ＪＫＬ）は右へふくらむ。外側のレーンほど大きくふくらむ。先の位置は `k × (0.55 + 0.45k)` で、だんだん速くなって勢いよく当たる。止めてもダメージは変わらない（ダメージはOnJudge で入るため）。

8. ロングの盾：張る・ヒビ・割れる（LongShield）

「ロングを押している間が暇」への答え。ロングの間、敵が盾を張る。曲がり角でうまく押し変えるたび、押し切った時にヒビが入り、全部入ると割れてダメージ。1つでも失敗すると、その盾は割れないだけで罰はない。

8-1. 判定する側（ChartObject::UpdateLongShield）

ChartObject.cpp　1334〜1375行

```
1334 void ChartObject::UpdateLongShield()
1335 {
1336     for (auto& n : m_Notes)
1337     {
1338         if (!n.IsLong() || n.shieldDone) continue;
1339 
1340         //先端が判定ラインに届いたら展開
1341         if (!n.shieldStarted)
1342         {
1343             if (m_MusicTime < n.arrivalTime) continue;
1344             n.shieldStarted = true;
1345             m_LongShield.BeginLong((int)n.turns.size() + 1);
1346         }
1347         //このフレームで終わったか
1348         bool ending = n.judged || m_MusicTime >= n.time;
1349 
1350         //曲がり角：曲がってからkTurnGrace秒たったら新しいレーンを押してるか調べる
1351         while (n.shieldTurnIndex < (int)n.turns.size())
1352         {
1353             const LaneTurn& tr = n.turns[n.shieldTurnIndex];
1354             bool onNewLane = m_AutoMode || Input::GetKeyPress(kLaneKeys[tr.lane]);    //新しいレーンを押しているか
1355 
1356             if (!ending)
1357             {
1358                 if (m_MusicTime < tr.time)break;    //まだ曲がり角に来ていない
1359                 if (!onNewLane && m_MusicTime < tr.time + JudgeSystem::kTurnGrace)break;    //まだ押し変えていないが猶
予がある
1360             }
1361             bool ok = ending ? n.shieldLastHeld : onNewLane;
1362             m_LongShield.Checkpoint(ok);
1363             n.shieldTurnIndex++;
1364         }
1365         //終点：最後のフレームで押せていたか
1366         if (ending)
1367         {
1368             m_LongShield.Checkpoint(n.shieldLastHeld);
1369             m_LongShield.EndLong();
1370             n.shieldDone = true;
1371             continue;
1372         }
1373         n.shieldLastHeld = IsHoldingLong(n);    //次のフレームのために今押せてるかを覚えておく
1374     }
1375 }
```

いつ	何をするか
先端が判定ラインに届いた	`BeginLong(曲がり角の数 + 1)`。ヒビが入るチャンスは「曲がり角ごと ＋ 終点」
曲がり角	曲がった時刻を過ぎたら、新しいレーンを押しているか調べる。押していなくても `kTurnGrace`（0.12秒）までは待つ
終点	「1つ前のフレームで押せていたか」（`shieldLastHeld`）で Checkpoint。そして EndLong

> **なぜこうしているか：**終点のフレームでは、ロングの判定がもう終わっていて、押していても「押している」と数えられないことがある。毎フレーム「今押せているか」を覚えておき、終わったフレームではその値を使うことで、最後まで押していた人が正しく成功になる。

8-2. 張る・数える・割る（LongShield.cpp）

LongShield.cpp 52〜72行

```
52 void LongShield::BeginLong(int checkpoints)
53 {
54     //盾がない　消えかけ・割れた直後も含む　新しい盾を張る
55     if (!IsUp())
56     {
57         m_State = State::Rising;
58         m_StateTime = 0.0f;
59         m_Active = 0;
60         m_Longs = 0;
61         m_Required = 0;
62         m_Done = 0;
63         m_Failed = false;
64         m_Cracks.clear();
65         m_Ripples.clear();
66         m_LastCrackAngle = Rand(0.0f, kTwoPi);
67     }
68     //盾が張られているときに始まったロングは同じ盾に足し合わせる
69     m_Active++;
70     m_Longs++;
71     m_Required += std::max(1, checkpoints);
72 }
```

LongShield.cpp 74〜95行

```
74 void LongShield::Checkpoint(bool success)
75 {
76     if (!IsUp()) return;
77     if (!success)
78     {
79         if (!m_Failed)
80         {
81             m_Failed = true;     //この盾はもう割れない
82             m_FailFlash = 1.0f;
83             if (m_OnFail) m_OnFail();
84         }
85         return;
86     }
87     if (m_Failed) return;     //失敗した後はひびを入れない
88
89     m_Done++;
```

```
90      AddCrack();
91      m_HitPulse = 1.0f;
92      if (m_OnCrack) m_OnCrack(m_Done, m_Required);
93
94      if (m_Done >= m_Required) Break();    //パフェ
95  }
```

LongShield.cpp 97〜106行

```
97  void LongShield::EndLong()
98  {
99      m_Active = std::max(0, m_Active - 1);
100     if (m_Active == 0 && IsUp())
101     {
102         //割れないままロングが終わったら消滅
103         m_State = State::Fading;
104         m_StateTime = 0.0f;
105     }
106 }
```

LongShield.cpp 108〜114行

```
108 void LongShield::Break()
109 {
110     m_State = State::Breaking;
111     m_StateTime = 0.0f;
112     SpawnShards();
113     if (m_OnBreak) m_OnBreak(m_Longs, GetCenter());
114 }
```

状態	いつ	見た目
Down	盾なし	何も出ない
Rising	BeginLong から 0.18秒（`kRiseTime`）	少し大きく出てから戻る
Up	張られている	六角形の縁とハチの巣。ヒビが入るたびに少し震える
Breaking	ヒビが全部入った	0.15秒で外へ弾けて消え、破片が飛ぶ
Fading	割れないままロングが全部終わった	0.3秒で静かに消える

> **なぜこうしているか：**盾が張られている間に別のロングが始まったら、新しい盾を張らずに、同じ盾に「必要なヒビの数」を足す。ロングが重なっても盾は1枚のまま。割れた時のダメージは「関わったロングの本数」分（OnBreak に m_Longs を渡す）。

8-3. 割れた後に ChartObject がすること

ChartObject.cpp 272〜282行

```
272     m_LongShield.SetOnBreak([this](int longCount, const Vector3& center)
273         {
274             GameEvent::OnShieldBreak().Publish({ center });
275             if (m_EnemyRef && m_LongCount > 0) m_EnemyRef->Damage(40.0f / m_LongCount * longCount);
276             if (m_EnemyRef) m_EnemyRef->Flash(Vector3(0.6f, 0.85f, 1.0f), 0.8f, 0.25f);
277             m_ImpactFx.SpawnCritical(center, Vector3(0.45f, 0.75f, 1.0f), 0.6f);
278             m_ScreenEffects.TriggerScreenShake(0.25f, 8.0f);
```

```
279             m_ScreenEffects.TriggerFlash(0.12f);
280             m_GazeKickTime = m_MusicTime;    //世界側のカメラを一瞬見上げる（今まで押し切りでやっていたもの）
281             sound.Play(SOUND_MIRROR_BREAK_BIG);    //割れた音
282         });
```

ChartObject.cpp　185〜192行　—　押し切った時（ダメージ・見上げる）

```
185     m_JudgeSystem.SetOnLongEnd([this](int lane)
186         {
187             GameEvent::OnLongEnd().Publish({ lane });
188             m_GazeKickTime = m_MusicTime;
189             //カウンター　HPの4割りをロングの本数で割った分のダメージにする
190             if (m_EnemyRef && m_LongCount > 0) m_EnemyRef->Damage(40.0f / m_LongCount);
191             if (m_EnemyRef) m_EnemyRef->Flash(Vector3(1.0f, 1.0f, 1.0f), 0.6f, 0.15f);
192         });
```

8-4. 盾の上の座標：2Dで考えて、最後に3Dへ

LongShield.cpp　120〜127行

```
120 void LongShield::Basis(Vector3& right, Vector3& up, Vector3& toCam) const
121 {
122     Vector3 camPos = m_HasCam ? m_CamPos : m_Center + Vector3(0.0f, 0.0f, -1000.0f);
123     RibbonShapes::CameraBasis(m_Center, camPos, right, up);
124     toCam = camPos - m_Center;
125     if (toCam.LengthSquared() < 0.0001f) toCam = Vector3(0.0f, 0.0f, -1.0f);
126     toCam.Normalize();
127 }
```

LongShield.cpp　135〜141行

```
135 Vector3 LongShield::ToWorld(float x, float y, float z) const
136 {
137     Vector3 right, up, toCam;
138     Basis(right, up, toCam);
139     Vector3 c = m_Center + up * kLift + toCam * kFront;
140     return c + right * (x * kStretchx) + up * y + toCam * z;
141 }
```

LongShield.cpp　144〜150行　—　盾の下の方を薄くする

```
144 float LongShield::VerticalFade(float y) const
145 {
146     float t = (y / kRadius - kFadeBottom) / (kFadeTop - kFadeBottom);    //下端０　上１
147     t = std::clamp(t, 0.0f, 1.0f);
148     t = t * t * (3.0f - 2.0f * t);    //スムーズ
149     return kFadeMin + (1.0f - kFadeMin) * t;
150 }
```

> **なぜこうしているか：**ヒビや六角形を3Dのまま計算すると難しい。そこで「盾の面の上の2D座標（x, y）」で全部を考え、描く直前に ToWorld で3Dに直している。面の向きは CameraBasis（外積で画面の右と上を出す）で決めるので、カメラが動いても盾はいつも正面を向く。盾の下の方は、レーンやノーツと重なりやすいので薄くしている。

8-5. ヒビを作る

LongShield.cpp 178〜213行

```
178 void LongShield::AddCrackLine(Crack& c, P2 start, float dir, float len, int depth)
179 {
180     //長さに合わせて決める。ガラスの日々はほぼまっすぐでところどころ小さく折れる
181     int steps = std::max(3, (int)(len / kRadius * 0.08f));
182     float step = len / (float)steps;
183 
184     std::vector<P2> line;
185     line.push_back(start);
186     P2 p = start;
187     int branchAt = (depth == 0 && Rand(0.0f, 1.0f) < 0.5f) ? (int)Rand(1.0f, (float)steps - 1.0f) : -1;
188 
189     for (int i = 0; i < steps; i++)
190     {
191         float kink = (Rand(0.0f, 1.0f) < 0.3f) ? Rand(-0.5f, 0.5f) : Rand(-0.12f, 0.12f);
192         dir += kink;
193         p.x += std::sin(dir) * step;
194         p.y += std::cos(dir) * step;
195 
196         float r = std::sqrt(p.x * p.x + p.y * p.y);
197         if (r > kRadius * 0.95f)    //盾の縁で止める
198         {
199             p.x *= kRadius * 0.95f / r;
200             p.y *= kRadius * 0.95f / r;
201             line.push_back(p);
202             break;
203         }
204         line.push_back(p);
205 
206         if (i == branchAt)    //枝分かれ
207         {
208             float side = (Rand(0.0f, 1.0f) < 0.5f) ? -1.0f : 1.0f;
209             AddCrackLine(c, p, dir + side * Rand(0.4f, 0.9f), len * Rand(0.25f, 0.45f), depth + 1);
210         }
211     }
212     c.lines.push_back(line);
213 }
```

LongShield.cpp 178〜213行

```
178 void LongShield::AddCrackLine(Crack& c, P2 start, float dir, float len, int depth)
179 {
180     //長さに合わせて決める。ガラスの日々はほぼまっすぐでところどころ小さく折れる
181     int steps = std::max(3, (int)(len / kRadius * 0.08f));
182     float step = len / (float)steps;
183 
184     std::vector<P2> line;
185     line.push_back(start);
186     P2 p = start;
187     int branchAt = (depth == 0 && Rand(0.0f, 1.0f) < 0.5f) ? (int)Rand(1.0f, (float)steps - 1.0f) : -1;
188 
189     for (int i = 0; i < steps; i++)
190     {
191         float kink = (Rand(0.0f, 1.0f) < 0.3f) ? Rand(-0.5f, 0.5f) : Rand(-0.12f, 0.12f);
192         dir += kink;
193         p.x += std::sin(dir) * step;
194         p.y += std::cos(dir) * step;
195 
196         float r = std::sqrt(p.x * p.x + p.y * p.y);
197         if (r > kRadius * 0.95f)    //盾の縁で止める
198         {
```

```
            p.x *= kRadius * 0.95f / r;
            p.y *= kRadius * 0.95f / r;
            line.push_back(p);
            break;
        }
        line.push_back(p);

        if (i == branchAt)    //枝分かれ
        {
            float side = (Rand(0.0f, 1.0f) < 0.5f) ? -1.0f : 1.0f;
            AddCrackLine(c, p, dir + side * Rand(0.4f, 0.9f), len * Rand(0.25f, 0.45f), depth + 1);
        }
    }
    c.lines.push_back(line);
}
```

- **起点は黄金角（約137.5度）ずつ回す：**ランダムに選ぶと同じ場所に固まることがある。黄金角ずつ回すと、何本入れても盾の全体に均等に散らばる（ひまわりの種の並びと同じ考え方）。
- **腕：**起点から3〜4本を放射状に。必要なヒビが多い盾では2〜3本にして線だらけにしない。
- **ぎざぎざ：**少しずつ進みながら向きを小さく曲げ、3割の確率で大きめに折れる。ガラスのヒビは「ほぼまっすぐで、ところどころ折れる」ので。
- **枝分かれ：**自分自身をもう一度呼ぶ（再帰）。`depth` で深さを数え、枝からさらには分かれない。
- **蜘蛛の巣：**隣の腕どうしを、少しずらした線でつなぐ（6割の確率）。
- **縁で止める：**盾の半径の 95% に着いたら止める。

8-6. 波紋と破片

LongShield.cpp　159〜176行

```
void LongShield::Ripple(const Vector3& worldPos)
{
    if (!IsUp()) return;
    if (m_Time - m_LastRipple < kRippleInterval) return;    //これ以上出さない
    m_LastRipple = m_Time;

    Vector3 right, up, toCam;
    Basis(right, up, toCam);
    Vector3 d = worldPos - GetCenter();
    RippleFx r;
    r.pos.x = d.Dot(right);    //右へどれだけか
    r.pos.y = d.Dot(up);    //上へどれだけか
    r.size = 28.0f;
    r.life = 0.3f;
    r.color = kShieldColor;
    if (m_Ripples.size() >= 12) m_Ripples.erase(m_Ripples.begin());
    m_Ripples.push_back(r);
}
```

当たった場所（3D）を、`right`・`up` との内積で盾の面の上の2D座標に直してから置く。0.1秒に1つまで、同時に12個まで。

LongShield.cpp　263〜284行

```
void LongShield::SpawnShards()
{
    for (int i = 0; i < kShardCount; i++)
    {
```

```
        float a = Rand(0.0f, kTwoPi);
        float r = std::sqrt(Rand(0.0f, 1.0f)) * kRadius * 0.85f;
        Shard s;
        s.x = std::sin(a) * r;
        s.y = std::cos(a) * r;
        s.z = 0.0f;
        float speed = Rand(250.0f, 480.0f);
        s.vx = std::sin(a) * speed;        //中心から外へ
        s.vy = std::fabs(std::cos(a)) * speed * 0.5f + Rand(120.0f, 300.0f);    //上へ
        s.vz = Rand(-260.0f, -60.0f);    //カメラの方へ
        s.rot = Rand(0.0f, kTwoPi);
        s.spin = Rand(-9.0f, 9.0f);
        s.size = Rand(18.0f, 48.0f);
        s.age = 0.0f;
        s.life = Rand(0.6f, 1.0f);
        m_Shards.push_back(s);
    }
}
```

18個の三角形が、中心から外へ・必ず上へ・カメラの方へ飛び、回りながら少し落ちて 0.6〜1秒で消える。破片が下へ落ちるとレーンに重なるので、必ず上と奥へ飛ばしている。

8-7. 描く（LongShield::Draw）

LongShield.cpp　345〜407行　—　前半：大きさ・色・縁・ハチの巣

```
void LongShield::Draw(Camera* cam)
{
    m_CamPos = cam->GetPosition();
    m_HasCam = true;

    bool showShield = (m_State == State::Rising || m_State == State::Up || m_State == State::Fading
        || (m_State == State::Breaking && m_StateTime < kBreakTime));
    if (!showShield && m_Shards.empty()) return;

    cam->SetCamera(0);
    Renderer::SetBlendState(BS_ADDITIVE);
    m_Pts.clear();

    if (showShield)
    {
        //大きさと濃さ
        float scale = 1.0f, alpha = 1.0f;
        if (m_State == State::Rising)
        {
            //少し大きく出てから戻る
            float t = std::clamp(m_StateTime / kRiseTime, 0.0f, 1.0f);
            scale = 1.0f + 0.15f * std::sin(t * RibbonShapes::kPi) - 0.6f * (1.0f - t) * (1.0f - t);
            alpha = t;
        }
        else if (m_State == State::Fading)
        {
            float f = std::clamp(m_StateTime / kFadeTime, 0.0f, 1.0f);
            scale = 1.0f + 0.08f * f;
            alpha = 1.0f - f;
        }
        else if (m_State == State::Breaking)
        {
            float f = std::clamp(m_StateTime / kBreakTime, 0.0f, 1.0f);
            scale = 1.0f + 0.25f * f;    //割れた瞬間、外へ弾けて消える
            alpha = 1.0f - f;
        }
        scale *= 1.0f + 0.04f * m_HitPulse;    //ヒビが入った瞬間に少し震える
```

```

        //色
        Vector3 col = m_Failed ? kFailColor : kShieldColor;
        col = col * (1.0f - m_FailFlash) + Vector3(1.0f, 0.3f, 0.25f) * m_FailFlash;
        float glowI = m_Failed ? 0.7f : 1.0f;

        const float R = kRadius * scale;

        //外側の太い縁と、ぼんやりした光
        AddHex(0.0f, 0.0f, R, 0.0f, col, alpha);
        Flush(cam, 10.0f, 1.3f * glowI, col);
        AddHex(0.0f, 0.0f, R * 0.96f, 0.0f, col, alpha * 0.25f);
        Flush(cam, 28.0f, 0.6f * glowI, col);

        //中のハチの巣（真ん中に1つ、まわりに6つ）
        float a = R * 0.3f;                    //小さい六角形の大きさ
        float d = a * 1.7320508f;              //隣の中心までの距離（√3 × 大きさ）
        AddHex(0.0f, 0.0f, a, 0.0f, col, alpha * 0.35f);
        Flush(cam, 2.5f, 0.8f * glowI, col);
        for (int k = 0; k < 6; k++)
        {
            float ang = kTwoPi * (k / 6.0f) + RibbonShapes::kPi / 6.0f;    //30度ずらした6方向
            AddHex(std::sin(ang) * d, std::cos(ang) * d, a, 0.0f, col, alpha * 0.35f);
            Flush(cam, 2.5f, 0.8f * glowI, col);
        }

```

LongShield.cpp　408〜473行　—　後半：ヒビ・波紋・破片

```

        //ヒビ(入った直後は太く明るい)
        for (const auto& c : m_Cracks)
        {
            float grow = std::clamp(c.age / kCrackGrow, 0.0f, 1.0f);
            float fresh = std::max(0.0f, 1.0f - c.age * 3.0f);
            for (const auto& line : c.lines)
            {
                if (line.size() < 2) continue;
                float shown = grow * (float)(line.size() - 1);    //何区間目まで見せるか
                int full = (int)shown;

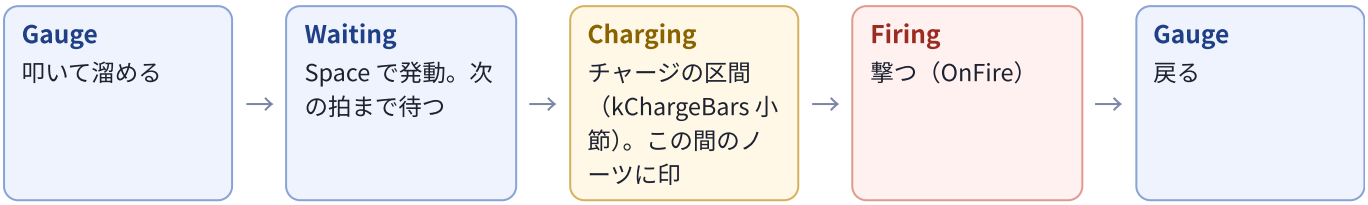
                for (int pass = 0; pass < 2; pass++)    //0：外側のぼんやりした光、1：細い芯
                {
                    if (pass == 0 && fresh <= 0.0f) continue;    //ぼんやりした光は入った直後だけ（描く回数を減らす）
                    for (size_t i = 0; i < line.size(); i++)
                    {
                        P2 p = line[i];
                        if ((int)i > full)    //伸びている途中：最後の区間は途中まで
                        {
                            float f = shown - (float)full;
                            P2 q = line[i - 1];
                            p.x = q.x + (p.x - q.x) * f;
                            p.y = q.y + (p.y - q.y) * f;
                        }
                        float taper = 1.0f - 0.65f * (i / (float)(line.size() - 1));    //先へ行くほど薄く
                        LongBeamRibbon::Point pt;
                        pt.pos = ToWorld(p.x * scale, p.y * scale);
                        pt.color = RibbonShapes::MakeColor(kCrackColor, alpha * taper * VerticalFade(p.y));
                        m_Pts.push_back(pt);
                        if ((int)i > full) break;
                    }
                    if (pass == 0) Flush(cam, 10.0f, 0.5f * fresh, kShieldColor);
                    else           Flush(cam, 2.5f + 2.0f * fresh, 1.3f + 0.8f * fresh, kShieldColor);
                }
            }
        }
```

```
444
445         //はじいた所の波紋（小さい六角形が広がって消える）
446         for (const auto& r : m_Ripples)
447         {
448             float k = r.age / 0.35f;
449             AddHex(r.pos.x, r.pos.y, 10.0f + 35.0f * k, 0.0f, kShieldColor, alpha * (1.0f - k));
450             Flush(cam, 5.0f, 1.2f, kShieldColor);
451         }
452     }
453
454     //破片：小さい三角形が回りながら飛ぶ
455     for (const auto& s : m_Shards)
456     {
457         float fade = 1.0f - std::clamp(s.age / s.life, 0.0f, 1.0f);
458         for (int i = 0; i <= 3; i++)
459         {
460             float ang = s.rot + kTwoPi * (i / 3.0f);
461             float y = s.y + std::cos(ang) * s.size;
462             LongBeamRibbon::Point q;
463             q.pos = ToWorld(s.x + std::sin(ang) * s.size, y, s.z);
464             q.color = RibbonShapes::MakeColor(Vector3(0.8f, 0.92f, 1.0f), fade * VerticalFade(y));
465             m_Pts.push_back(q);
466         }
467         float w = kShaderFilled ? s.size * 0.2f : 300.5f;            // 塗りつぶし風：辺の線を太くして中を埋める
468         float lightI = kShaderFilled ? 0.8f : 1.5f;            // 太いと明るすぎるので少し下げる
469         Flush(cam, w, lightI, kShieldColor);
470     }
471
472     Renderer::SetBlendState(BS_NONE);
473 }
```

- 張られる時：少し大きく出てから戻る（sin で膨らみ、2乗で縮む）。消える時・割れる時は、少し大きくなりながら薄くなる。
- ハチの巣：真ん中に1つ、まわりに6つ。小さい六角形の大きさ a に対して、隣の中心までは a × $\sqrt{3}$（1.7320508）。この距離で並べると、すき間なく並ぶ。
- ヒビは入ってから 0.08秒（`kCrackGrow`）かけて伸びる。入った直後だけ太く明るく（`fresh`）、先へ行くほど薄く（`taper`）。
- 失敗した盾は灰色。失敗した瞬間だけ赤く光る（`m_FailFlash`）。

9. 必殺技：ゲージ・弱点・ビーム

9-1. SpecialMove：状態と計算だけ



SpecialMove.cpp 22〜47行

```
22 bool SpecialMove::TryActivate(float musicTime, float bpm, int beatsPerBar, std::vector<ChartNote>& notes)
23 {
24     if (!IsReady()) return false;
25     if (musicTime < 0.0f) return false;        // カウントイン中は発動しない
26 
27     float beat = (bpm > 0.0f) ? 60.0f / bpm : 0.5f;
28     float bar = beat * (float)std::max(1, beatsPerBar) * kChargeBars;
29     m_ChargeStart = std::ceil(musicTime / beat) * beat;    // 次の拍（曲の0秒が1拍目の前提）
30     m_ChargeEnd = m_ChargeStart + bar;
31 
32     m_ChargeWeight = 0.0f;
33     m_ChargeNoteCount = 0;
34     for (auto& n : notes)
35     {
36         if (n.judged || n.IsLong()) continue;        // v1 はタップとフリックだけ
37         if (n.time >= m_ChargeStart && n.time < m_ChargeEnd)
38         {
39             n.chargeShot = true;
40             m_ChargeNoteCount++;
41         }
42     }
43 
44     m_Gauge = 0.0f;
45     SetState(State::Waiting, musicTime);
46     return true;
47 }
```

SpecialMove.cpp 49〜82行

```
49 void SpecialMove::Update(float deltaTime, float musicTime)
50 {
51     switch (m_State)
52     {
53     case State::Gauge:
54         break;
55 
56     case State::Waiting:
57         if (musicTime >= m_ChargeStart)
58         {
59             SetState(State::Charging, musicTime);
60             if (m_OnChargeStart) m_OnChargeStart();
61         }
62         break;
63 
64     case State::Charging:
```

```
65          if (musicTime >= m_ChargeEnd)
66          {
67              m_FireTimer = 0.0f;
68              SetState(State::Firing, musicTime);
69              if (m_OnFire) m_OnFire(GetChargeRatio());
70          }
71          break;
72
73      case State::Firing:
74          m_FireTimer += deltaTime;
75          if (m_FireTimer >= kFireDuration)
76          {
77              SetState(State::Gauge, musicTime);
78              if (m_OnFinish) m_OnFinish();
79          }
80          break;
81      }
82  }
```

SpecialMove.cpp 84〜96行

```
84  void SpecialMove::AddHit(int rank, bool isChargeNote)
85  {
86      if (isChargeNote)
87      {
88          // チャージノーツは少し早く叩くと Waiting 中に判定されることがあるので、どちらでも数える
89          if (m_State == State::Waiting || m_State == State::Charging)
90              m_ChargeWeight += RankWeight(rank);
91          return;
92      }
93      // 溜め中だけゲージが増える（発動中は増えない）
94      if (m_State == State::Gauge && m_TotalNote > 0)
95          m_Gauge = std::min(1.0f, m_Gauge + 1.0f / (kFillRatio * (float)m_TotalNote));
96  }
```

- ゲージ：1回叩くごとに `1 ÷ (0.4 × 総ノーツ数)` 溜まる。総ノーツの4割（`kFillRatio`）で満タン。判定の良し悪しは関係なし。
- チャージ量：チャージ中のノーツの重み（WEAK 1.0／HIT 0.6／GRAZE 0.3）の平均（0〜1）。
- ダメージの式 `CalcDamage = 基本値 × (0.3 + 0.7 × チャージ量)`。全部取り逃しても3割は入る。

> **なぜこうしているか：**SpecialMove は見た目もダメージも持たず、「状態が変わった」をコールバックで知らせるだけ。JudgeSystem と同じ考え方で、ルールを変えても演出を、演出を変えてもルールを触らずに済む。

9-2. 発動と発射（ChartObject）

ChartObject.cpp 1158〜1172行 — Space の処理（ChartObject::Update）

```
1158      if (Input::GetKeyTrigger(VK_SPACE))
1159      {
1160          if (m_Special.GetState() == SpecialMove::State::Gauge)
1161          {
1162              if (m_Special.TryActivate(m_MusicTime, m_Bpm, m_BeatsPerBar, m_Notes))
1163              {
1164                  m_JudgeSystem.SetMissFressWindow(m_Special.GetChargeStart(), m_Special.GetChargeEnd());
1165                  m_WeakPoint.BeginSpecial(m_MusicTime, m_Special.GetChargeEnd(), m_LastPressedLane);
1166              }
1167          }
```

```
1168        else if (m_WeakPoint.Lock())
1169        {
1170            sound.Play(SOUND_BEAM_DECITION);    //ロックオンの音
1171        }
1172    }
```

ChartObject.cpp　158〜178行　—　撃つ時（ChartObject::Init）

```
158    m_Special.SetOnFire([this](float ratio)
159        {
160            ChargeAura::Instance().Release();
161            if (!m_EnemyRef) return;
162
163            //オートプレイは必ず弱点を狙う
164            if (m_AutoMode) m_WeakPoint.SetAim(m_WeakPoint.GetWeakLane());
165
166            int aim = m_WeakPoint.GetAim();
167            bool critical = (aim == m_WeakPoint.GetWeakLane());
168
169            constexpr float kFromUp = 40.0f;    //プレイヤーのどれだけ上から撃つか
170            constexpr float kToUp = 50.0f;        //敵の中心のどれだけ上に当てるか
171            Vector3 from = m_PlayerRef ? m_PlayerRef->GetPosition() + Vector3(0.0f, kFromUp, 0.0f)
172                : Vector3(0.0f, 0.0f, -300.0f);
173            Vector3 to = m_EnemyRef->GetPosition() + Vector3(0.0f, kToUp, 0.0f);    //ビームは必ず敵のど真ん中へ
174
175            m_WeakPoint.OnFired();
176            m_SpecialBeam.Fire(from, to, ratio, critical);    //ダメージは当たった瞬間（SetOnHit）
177            sound.Play(SOUND_BEAM_FIRING);
178        });
```

ChartObject.cpp　286〜311行　—　当たった時（ChartObject::Init）

```
286    m_SpecialBeam.SetOnHit([this](float ratio, bool critical, const Vector3& hitPos)
287        {
288            float damage = SpecialMove::CalcDamage(kSpecialBaseDamage, ratio)
289                * (critical ? kSpecialCritMul : kSpecialBlockedMul);
290            if (m_EnemyRef) m_EnemyRef->Damage(damage);
291
292            if (critical)
293            {
294                //弱点に当たった：大きな衝撃・閃光・強い揺れ・世界側カメラのキック
295                m_ImpactFx.SpawnCritical(hitPos, Vector3(1.0f, 0.85f, 0.4f), ratio);
296                m_ScreenEffects.TriggerScreenShake(0.35f, 8.0f + 10.0f * ratio);
297                m_ScreenEffects.TriggerFlash(0.18f);
298                if (m_EnemyRef) m_EnemyRef->Flash(Vector3(1.0f, 0.9f, 0.5f), 1.0f, 0.35f);    //金色に強く
299                m_GazeKickTime = m_MusicTime;
300                sound.Play(SOUND_METAL_2);
301            }
302            else
303            {
304                //防がれた：盾と、はね返る火花 盾がいるかは要検討
305                Vector3 from = m_PlayerRef ? m_PlayerRef->GetPosition() : Vector3(0.0f, 0.0f, -300.0f);
306                m_ImpactFx.SpawnBlocked(hitPos, hitPos - from);
307                m_ScreenEffects.TriggerScreenShake(0.15f, 4.0f);
308                if (m_EnemyRef) m_EnemyRef->Flash(Vector3(0.55f, 0.75f, 1.0f), 0.6f, 0.2f);    //青白く（盾の色があったら）
309                sound.Play(SOUND_REFLECTION);
310            }
311        });
```

9-3. WeakPoint：弱点は必殺技の時だけ

WeakPoint.cpp 154〜162行

```
154 void WeakPoint::BeginSpecial(float musicTime, float fireTime, int startAim)
155 {
156     m_Mode = Mode::Aiming;
157     PickNew();                    // 弱点を選んで出す。撃つまで固定
158     m_PhaseStart = musicTime;
159     m_PhaseEnd = fireTime;        // 残り時間の輪は「撃つまで」を表す
160     m_Locked = false;
161     m_Aim = std::clamp(startAim, 0, 5);
162 }
```

WeakPoint.cpp 104〜121行

```
104 void WeakPoint::PickNew()
105 {
106     std::uniform_int_distribution<int> laneDist(0, 5);
107     int lane = laneDist(m_Rng);
108 
109     float x = 0.0f, y = 0.0f;
110     for (int tries = 0; tries < 12; tries++)
111     {
112         RandomOffset(x, y);
113         float dx = x - m_OffX, dy = y - m_OffY;
114         if (dx * dx + dy * dy >= kPosMinMove * kPosMinMove) break;    // 前の場所から十分離れた
115     }
116 
117     m_WeakLane = lane;
118     m_OffX = x;
119     m_OffY = y;
120     m_SwitchFlash = 1.0f;
121 }
```

必殺技を発動した時に、弱点のキー（S〜L）と敵の上の場所をランダムに選び、撃つまで固定する。前の場所から少なくとも 90 離す（12回まで選び直す）。レーンのキーで狙いを動かし、もう一度 Space でロック。狙いが弱点なら金色の枠になる。普段のノーツは譜面で決まっていて狙えないので、弱点は必殺技の時だけにした。

9-4. SpecialBeam：3層のビーム

SpecialBeam.cpp 76〜128行

```
76 void SpecialBeam::Fire(const Vector3& from, const Vector3& to, float chargeRatio, bool critical)
77 {
78     m_From = from;
79     m_To = to;
80     m_Ratio = std::clamp(chargeRatio, 0.0f, 1.0f);
81     m_Critical = critical;
82     m_Time = 0.0f;
83     m_Active = true;
84     m_HitSent = false;
85 
86     //進む向きと、それに対する「横」「上」
87     Vector3 d = to - from;
88     float len = d.Length();
89     if (len < 1.0f) { len = 1.0f; d = Vector3(0.0f, 0.0f, 1.0f); }
90     d.Normalize();
91     m_AxisV = Vector3::Up.Cross(d);
92     if (m_AxisV.LengthSquared() < 0.0001f) m_AxisV = Vector3(1.0f, 0.0f, 0.0f);
```

```
    m_AxisV.Normalize();
    m_AxisU = d.Cross(m_AxisV);
    m_AxisU.Normalize();

    //群れ：1本ずつ、ふくらむ向き,大きさ,遅れ.速さ.太さ.色を変える
    //向きは「真上を0」として ±kSwarmSpreadDeg の範囲を本数で均等に分け、少しだけずらす
    //（ただのランダムだと偏ることがあるので、均等＋ずらし にしている）
    m_Strands.clear();
    const float spread = kSwarmSpreadDeg * RibbonShapes::kPi / 180.0f;
    std::uniform_int_distribution<int> pick(0, kPaletteCount - 1);
    for (int i = 0; i < kSwarmCount; i++)
    {
        float slot = (i + Rand(0.0f, 1.0f)) / (float)kSwarmCount;    //0〜1
        float ang = -spread + 2.0f * spread * slot;
        Vector3 off = m_AxisU * std::cos(ang) + m_AxisV * std::sin(ang);

        float ang2 = ang + Rand(-0.8f, 0.8f);    //当てる側は少し違う向きにして、ねじれた弧にする
        Vector3 off2 = m_AxisU * std::cos(ang2) + m_AxisV * std::sin(ang2);

        float r1 = Rand(kSwarmRadiusMin, kSwarmRadiusMax);
        float r2 = r1 * Rand(0.2f, 0.5f);

        Strand s;
        s.c1 = from + d * (len * Rand(0.10f, 0.35f)) + off * r1;
        s.c2 = to - d * (len * Rand(0.10f, 0.30f)) + off2 * r2;
        s.delay = Rand(0.0f, kSwarmDelayMax);
        s.grow = Rand(kSwarmGrowMin, kSwarmGrowMax);
        s.width = Rand(kSwarmWidthMin, kSwarmWidthMax) * (0.6f + 0.4f * m_Ratio);
        s.color = kSwarmPalette[pick(m_Rng)];
        s.arrived = false;
        m_Strands.push_back(s);
    }

    m_BoltTimer = 0.0f;
    RefreshBolts();
}
```

SpecialBeam.cpp　147〜157行　—　本命の帯が届いた瞬間に1回だけ知らせる

```
void SpecialBeam::Update(float deltaTime)
{
    if (!m_Active) return;
    m_Time += deltaTime;

    //最初の帯の先が敵に届いた瞬間に一回通知
    if (!m_HitSent && m_Time >= kMainGrow)
    {
        m_HitSent = true;
        if (m_OnHit)m_OnHit(m_Ratio, m_Critical, m_To);
    }
```

層	中身
本命の帯	太い金の帯（チャージ量で太さ 50〜140）。上にふくらむ弧。先が当たった瞬間に OnHit を1回
群れ	色とりどりの細い光 24本。向きは「均等に分けて少しずらす」（ただのランダムだと偏るため）。当たる側は向きを少し変えて、ねじれた弧にする。1本当たるたびに OnSubHit
雷	帯にまとわりつくギザギザの線 3本。0.045秒ごとに形を作り直すので、チカチカ動く

10. 全部を支える道具

10-1. LongBeamRibbon：点の並び → 光の帯

LongBeamRibbon.cpp　29〜86行

```
29 void LongBeamRibbon::Draw(Camera* cam, const std::vector<Point>& points, float width, float intensity,
30     float time, const Vector3& glowColor)
31 {
32     int n = std::min((int)points.size(), kMaxPoints);
33     if (n < 2) return;
34 
35     const Vector3 camPos = cam->GetPosition();
36     float v = 0.0f;     // 線に沿った距離（幅1つ分で1進む）
37 
38     for (int i = 0; i < n; i++)
39     {
40         // この点で線が進む向き（前後の点から求める）
41         Vector3 a = points[(i > 0) ? i - 1 : i].pos;
42         Vector3 b = points[(i < n - 1) ? i + 1 : i].pos;
43         Vector3 dir = b - a;
44         if (dir.LengthSquared() < 0.000001f) dir = Vector3(0.0f, 0.0f, 1.0f);
45         dir.Normalize();
46 
47         // 線に垂直で、カメラから見て横に広がる向き（外積）
48         Vector3 side = dir.Cross(camPos - points[i].pos);
49         if (side.LengthSquared() < 0.000001f) side = Vector3(1.0f, 0.0f, 0.0f);
50         side.Normalize();
51         side *= width * 0.5f;
52 
53         if (i > 0) v += (points[i].pos - points[i - 1].pos).Length() / width;
54 
55         VERTEX_3D& L = m_Vertices[i * 2 + 0];
56         VERTEX_3D& R = m_Vertices[i * 2 + 1];
57         L.position = points[i].pos - side;
58         R.position = points[i].pos + side;
59         L.color = R.color = points[i].color;
60         L.uv = Vector2(0.0f, v);
61         R.uv = Vector2(1.0f, v);
62     }
63     m_VertexBuffer.Modify(m_Vertices.data(), (size_t)n * 2);     // 使う分（点の数×2）だけ送る
64 
65     // 頂点はもうワールド座標なので、World行列は単位行列
66     Matrix world = Matrix::Identity;
67     Renderer::SetWorldMatrix(&world);
68 
69     Params p = {};
70     p.glowColor = Vector4(glowColor.x, glowColor.y, glowColor.z, 1.0f);
71     p.intensity = intensity;
72     p.time = time;
73     m_Shader.SetGPU();
74     m_Shader.UpdatePSConstantBuffer(&p, sizeof(p));
75     if (m_NoiseLoaded) m_Noise.SetGPU(0);
76     m_VertexBuffer.SetGPU();
77 
78     ID3D11DeviceContext* ctx = Renderer::GetDeviceContext();
79     ctx->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
80 
81     Renderer::SetCullMode(CULL_NONE);         // 帯の裏表を気にしない
82     Renderer::SetDepthEnable(false);         // 深度テストはするが書き込まない（半透明の光なので）
```

```
    ctx->Draw(n * 2, 0);
    Renderer::SetDepthEnable(true);
    Renderer::SetCullMode(CULL_BACK);
}
```

1. **点ごとに左右2つの頂点を作る：**線が進む向き `dir` と、カメラへの向きの外積で「横へ広げる向き」`side` を出す。両方に直角なので、帯はいつもカメラの方を向き、同じ太さに見える。
2. **線に沿った距離を UV に入れる：**横は 0〜1、縦は始まりからの距離。模様が流れ、つなぎ目が出ない。
3. **三角形ストリップで1回で描く：**L, R, L, R… と並べると、続く3つずつが三角形になる。
4. **使う分だけ GPU に送る：**点の数×2 の頂点だけを送る（下の VertexBuffer）。

shader/longBeamPS.hlsl 1〜35行 — 光らせるシェーダー

```
#include "common.hlsl"

Texture2D noiseTex : register(t0);
SamplerState samp : register(s0);

cbuffer LongBeamParams : register(b6)
{
    float4 glowColor;    // 外側の柔らかい光の色
    float intensity;    // 全体の明るさ
    float time;            // 模様を流す時間
    float2 padding;
}

// tex.x : 帯の横方向（0=左端 0.5=中心 1=右端）
// tex.y : 線に沿った距離（手前が0、奥ほど大きい）
float4 main(in PS_IN input) : SV_TARGET
{
    float d = abs(input.tex.x - 0.5) * 2.0;    // 中心0〜端1

    float core = exp(-d * d * 6.0);        // 細く強い芯
    float glow = exp(-d * d * 1.0);            // 太く柔らかい光

    // 線に沿って、奥から手前へ流れるエネルギーの揺らぎ
    float2 uv = float2(input.tex.x * 0.6, input.tex.y * 0.12 + time * 1.5);
    float n = noiseTex.Sample(samp, uv).r;
    float energy = 0.7 + 0.6 * n;

    float3 coreColor = lerp(input.col.rgb, float3(1.0, 1.0, 1.0), 0.6);    // 芯は白寄りのレーン色
    float3 color = coreColor * core * 1.8 + glowColor.rgb * glow * 0.8 * energy;
    color *= intensity;

    // 加算合成（色 × α）で使う。αは奥のフェード用
    return float4(color, input.col.a);
}

```

`exp(-d²×6)`（細く強い芯）と `exp(-d²)`（太く柔らかい光）を足し、ノイズの画像で揺らぎを付ける。芯には白を6割混ぜる。加算合成で重ねる。

> **なぜこうしているか：**点の並べ方を変えるだけで、円・六角形・曲線・ぎざぎざ・三角形・十字の光と、何でも描ける。テクスチャやモデルを用意しなくてよく、太さ・色・濃さも点ごとに決められる。

10-2. 使う分だけ頂点を送る（VertexBuffer.h）

VertexBuffer.h　63〜72行

```
63      void Modify(const T* data, size_t count)
64      {
65          D3D11_MAPPED_SUBRESOURCE msr;
66          HRESULT hr = Renderer::GetDeviceContext()->Map(
67              m_VertexBuffer.Get(), 0, D3D11_MAP_WRITE_DISCARD, 0, &msr);
68          if (SUCCEEDED(hr)) {
69              memcpy(msr.pData, data, count * sizeof(T));
70              Renderer::GetDeviceContext()->Unmap(m_VertexBuffer.Get(), 0);
71          }
72      }
```

以前は、点3つの帯でも毎回 1024個（最大数）の頂点を送っていた。使う分だけ送るようにしたので、帯を使う全部の演出（攻撃・盾・火花・ロング）が軽くなった。

10-3. RibbonShapes：点を形に並べる

RibbonShapes.h　17〜60行　—　CameraBasis・Arc・Polygon・MakeColor

```
17      inline void CameraBasis(const Vector3& center, const Vector3& camPos, Vector3& right, Vector3& up)
18      {
19          Vector3 n = camPos - center;    //カメラへ向かう向き
20          if (n.LengthSquared() < 0.00001f) n = Vector3(0.0f, 0.0f, -1.0f);
21          n.Normalize();
22
23          right = n.Cross(Vector3::Up);    //画面の右
24          if (right.LengthSquared() < 0.00001f) right = Vector3(1.0f, 0.0f, 0.0f);
25          right.Normalize();
26
27          up = right.Cross(n);    //画面の上
28          up.Normalize();
29      }
30
31      //円弧の点をoutに足す
32      //startRad:始まりの角度（真上：0）sweepRad:どれだけ回るか（2πで一周）
33      inline void Arc(Points& out, const Vector3& center, const Vector3& right, const Vector3& up,
34          float radius, float startRad, float sweepRad, int segments, const Color& color)
35      {
36          if (segments < 1) segments = 1;
37          for (int i = 0; i <= segments; i++)
38          {
39              float a = startRad + sweepRad * (i / (float)segments);
40              LongBeamRibbon::Point p;
41              p.pos = center + (right * std::sin(a) + up * std::cos(a)) * radius;
42              p.color = color;
43              out.push_back(p);
44          }
45      }
46
47      //正多角形の点をoutに足す。sides=6の六角形
48      inline void Polygon(Points& out, const Vector3& center, const Vector3& right, const Vector3& up,
49          float radius, int sides, float rotRad, const Color& color)
50      {
51          Arc(out, center, right, up, radius, rotRad, 2.0f * kPi, sides, color);
52      }
53
54      //Vector3の色と濃さからColorを作る
55      inline Color MakeColor(const Vector3& rgb, float alpha)
56      {
```

```
        return Color(rgb.x, rgb.y, rgb.z, alpha);
    }
}

```

11. 読んでいて気になった所

直すかどうかは判断してほしいもの。上ほど影響が大きい。

① enemyFlash.hlsl の戻り値の型が float（`shader/enemyFlash.hlsl : 12行`）
色は4つの値（RGBA）なので `float4 main(...)` が正しい。今の形だと、色の最初の1つ（赤）しか画面に書かれない可能性がある。敵が暗く・黒っぽく見えているなら、これが原因かもしれない。

② ロングで盾を割ると、ダメージが2回入る
押し切った時の SetOnLongEnd（40 ÷ ロングの本数）と、盾が割れた時の SetOnBreak（同じ量 × 本数）の両方が入る。割れた時だけにするか、片方を減らすか決めておくとよい。

③ 盾が張られている間のタップはダメージなし（`ChartObject.cpp : 1240行`）
最初のロングの間に入れたトレースや、最後のロングの間のタップは、ダメージにならない。狙い通りならそのままでよい。レーザーは盾に当たって波紋になる。

④ HitBurstFx の上限が小さい（kMaxBursts 4、kMaxLasers 10）
上限を超えると古いものから消す。消されたレーザーが「当たった」を知らせる役（notify）だった場合、着弾の演出が出ない。破片のかたまりも、撃つ前に消されることがある。

⑤ 必殺技がいつでも撃てる設定のまま（SpecialMove.h の kDebugAlwaysReady = true）
ゲージなしでいつでも発動できる。提出の前に false にするか決めておく。SpecialMove.h のコメントには「1小節」とあるが、チャージの長さは kChargeBars（4小節）。

⑥ トレースは判定の時刻を過ぎた最初のフレームで押していないとミス（`TraceNote.cpp : 35〜50行`）
少しでも押すのが遅れるとミスになる。「0.10秒過ぎても押していなければミス」に戻すなら、`else` を `else if (d > 0.10f)` にする。

⑦ 小さなこと

- LaneTrack.h の SpawnOffset は、Right が (900, **200**, 0)、Left が (−900, **0**, 0) で、上下がそろっていない。switch の最後に return がないので、コンパイラの警告が出ることがある。
- Enemy::Update は dt = 1/60 で固定。画面の更新が速い・遅いパソコンでは、回転や点滅の速さが変わる。
- RankColor の GRAZE のコメントは「紫」だが、値は緑 (0, 1, 0)。
- AttackOrbs は今使われていない（kTapAttackOrbs = false）。m_LastOrbTime も含めて、残すか消すか。
- WeakPoint.h の kBarsPerSwitch（4小節ごとに弱点を変える）は、今は使われていない（弱点は必殺技の時だけ出す形に変えたため）。
- OnLongEnd と OnShieldBreak のイベントは、放送しているが、まだ誰も聞いていない（拍ごとのレーザーや背景の演出を足す時に使える）。